

Inteligență Artificială

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

IA-INFO-24 - curs 4

Kahoot! Summary

Rank	Player	Total scor fără întrebarea 1	Total scor	Puncte întrebarea 1
1	241DorneanuD	3727	3727	0
2	SpiridonMihn244	3714	4137	423
3	242bejandiana	3704	3704	0
4	241preda	3664	3664	0
5	243StoianCristi	3636	3636	0
6	241 Minea S	3627	4018	391
7	244Rusu_Iulia	3593	3593	0
8	Tanasoiu_243	3544	3544	0
9	241Sandulescu	3504	3754	250
10	Ciocîrlan C 242	3497	3890	393
11	Leusteans241	3486	3885	399
12	243LoghinA	3486	3486	0
13	242Tanase	3475	3475	0
14	244Tanase	3454	3704	250
15	PupezescuA241	3449	3449	0
16	242VisanMiruna	3439	3439	0
17	244Riclea	3428	3428	0
18	244 panaet	3400	3400	0
19	ioana cocut 244	3381	3381	0
20	243RoscaTeo	3311	3311	0
21	241bulacuDaria	3283	3533	250
22	Hangu Calin 241	3216	3466	250
23	242Avram	3005	3005	0
24	ManeaBianca244	2756	2756	0
25	244DragunoiM	2739	3160	421
26	244 Iacob Alin	2716	2716	0
27	243GunaA	2665	2915	250
28	241UngureanuS	2604	2604	0
29	244NegruRaluca	2602	2602	0
30	243 Bizon Anca	2599	2849	250
31	243mitrachedani	2588	3002	414
32	241NedelcuA	2584	2584	0
33	244VoinescuA	2538	2538	0
34	ZahariaAris242	2402	2402	0
35	robituriana244	2362	2362	0
36	Olteanu 242	2358	2358	0
37	SavaDarius242	2310	2310	0
38	Neacsa242	2295	2295	0
39	241BarbulescuB	1754	1754	0
40	rachierugabi241	1663	1663	0
41	343 Shahsavari	1586	1586	0
42	FologeaS 241	972	972	0
43	sebiTica241	891	891	0

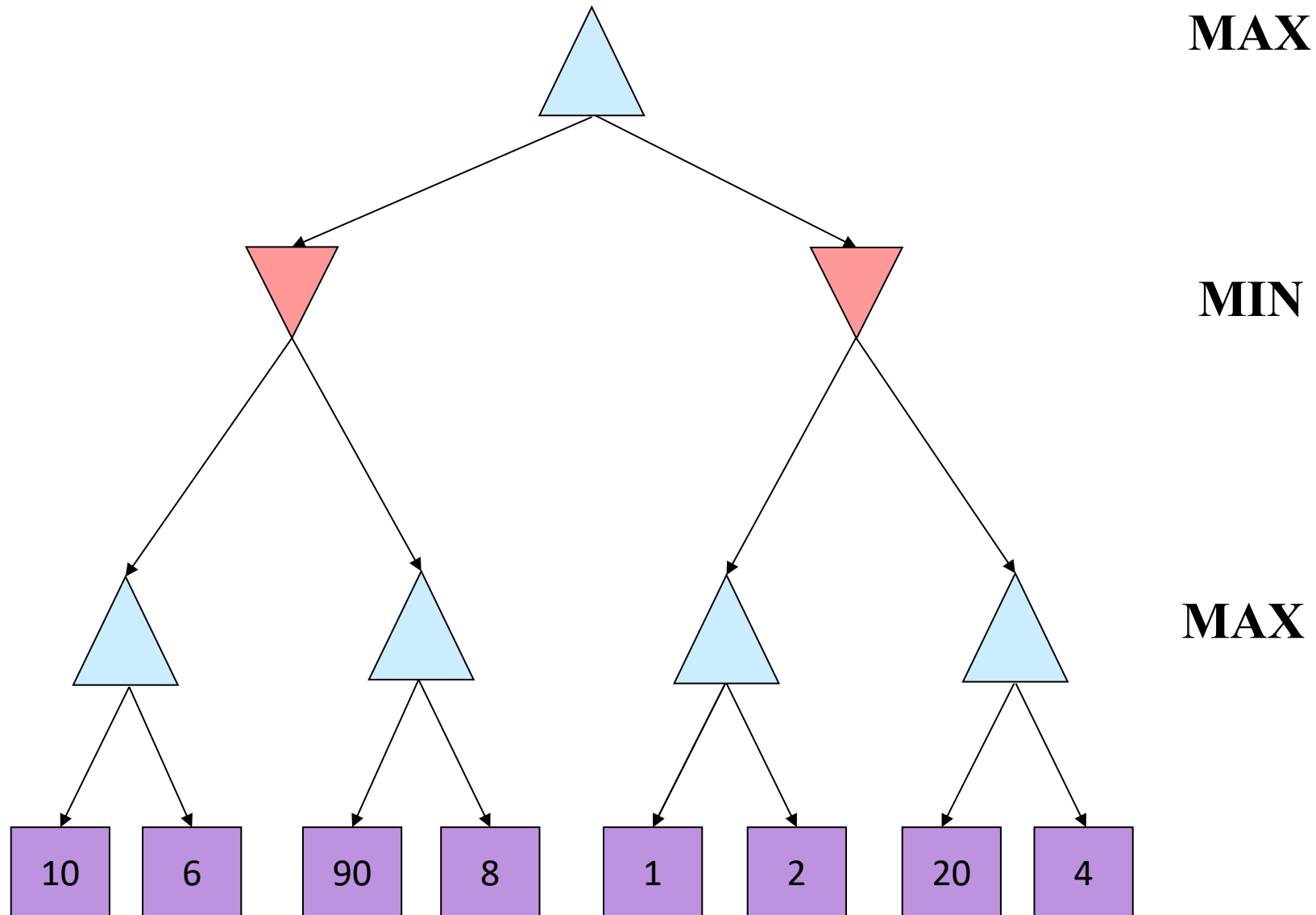
Recapitulare – cursul trecut

1. Crearea de euristici admisibile
2. Proprietățile și alte variante ale algoritmului A*
3. Căutare adversarială:
 - jocuri adversariale
 - algoritmul MINIMAX
 - algoritmul α - β retezare (α - β pruning)

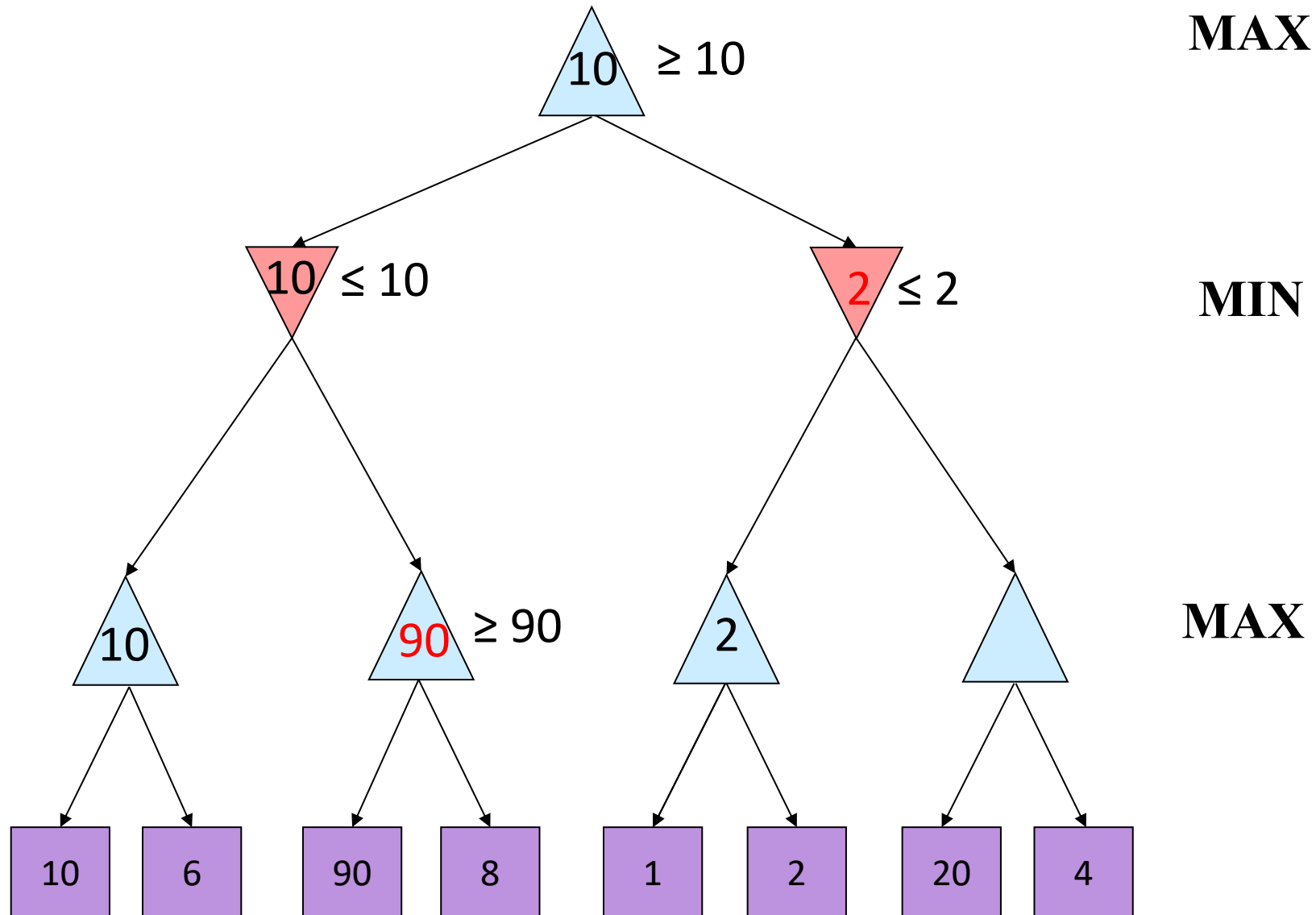
Cuprinsul cursului de azi

1. Căutare ExpectiMax
2. Probleme de satisfacere de constrângeri
3. Algoritmi de căutare locală
4. Algoritmi genetici

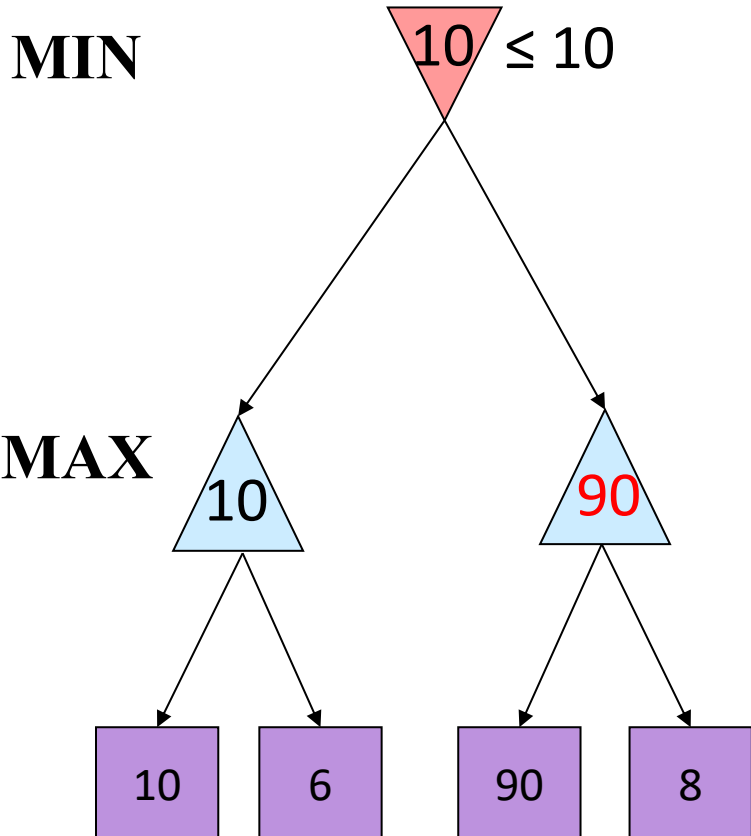
Exemplu



Exemplu

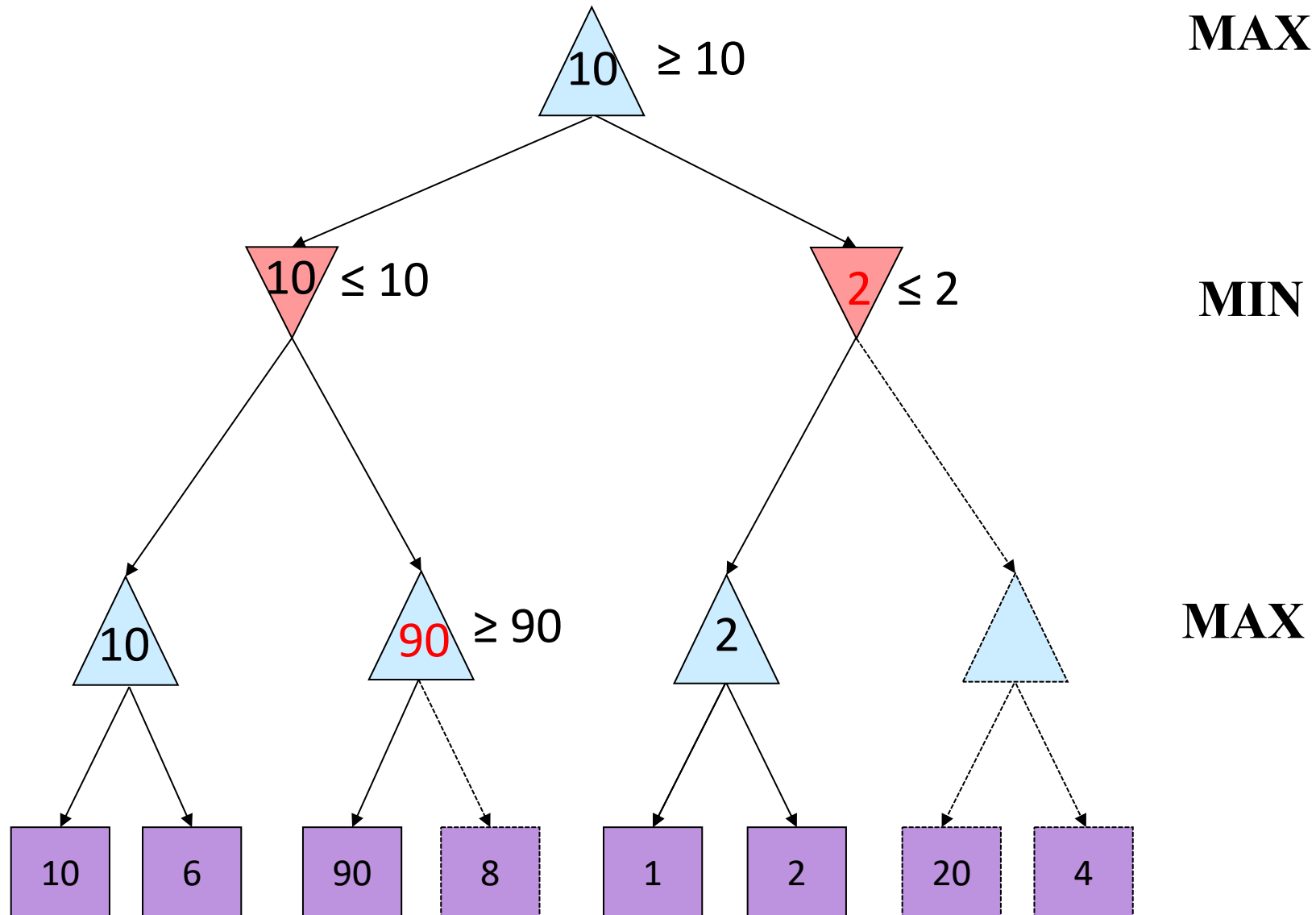


Exemplu alfa-beta retezare

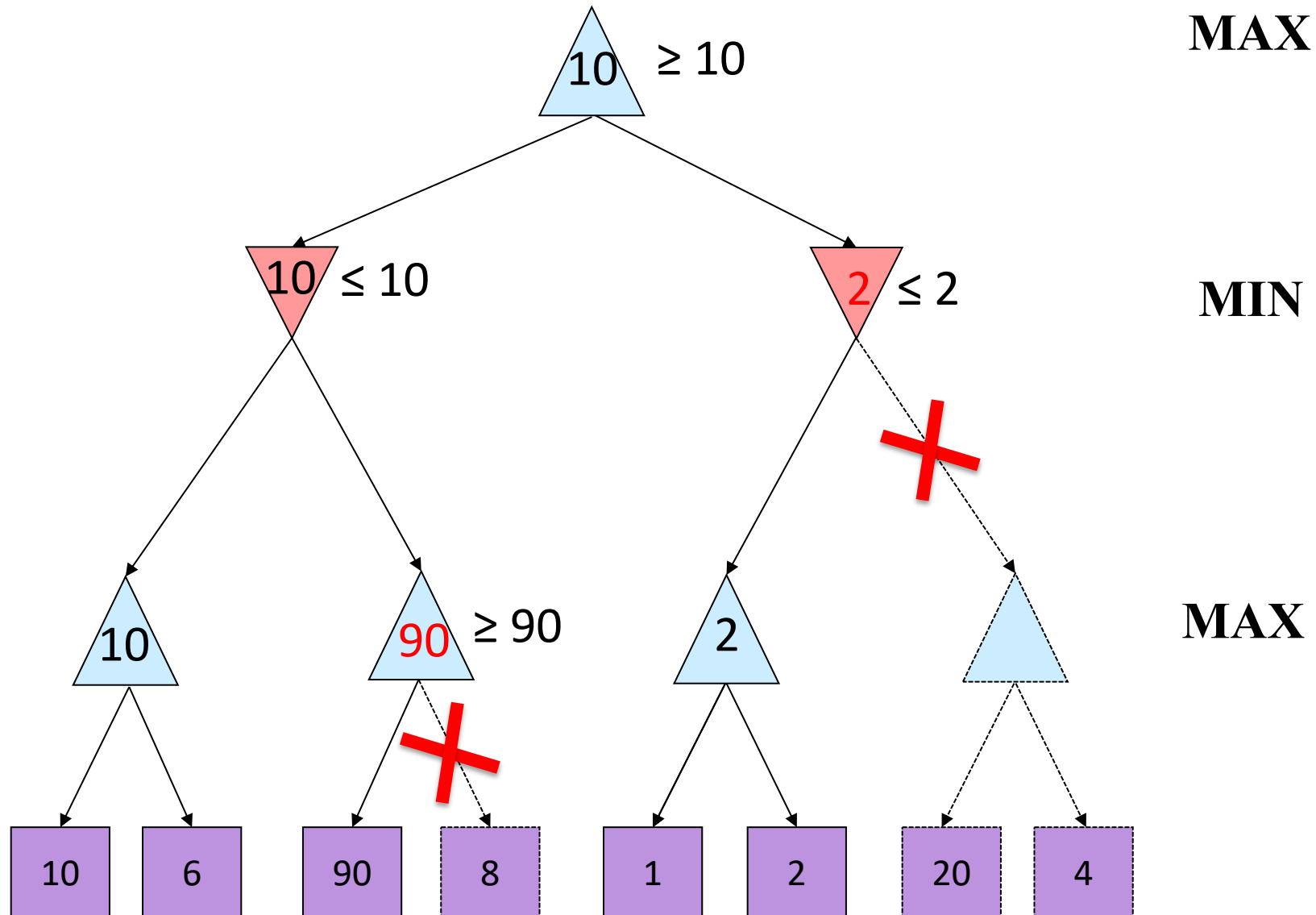


Pentru nodurile de tip MIN: după ce parcurg prima ramură din stânga (sub-arborele stâng) am o valoare maximă β . Pot atunci, pentru sub-arborii care urmează, să nu mai vizitez (să retez) sub-arbori din nodurile de tip MAX pentru care avem $\alpha \geq \beta$. Aproximez valoarea aceluia nod MAX cu α .

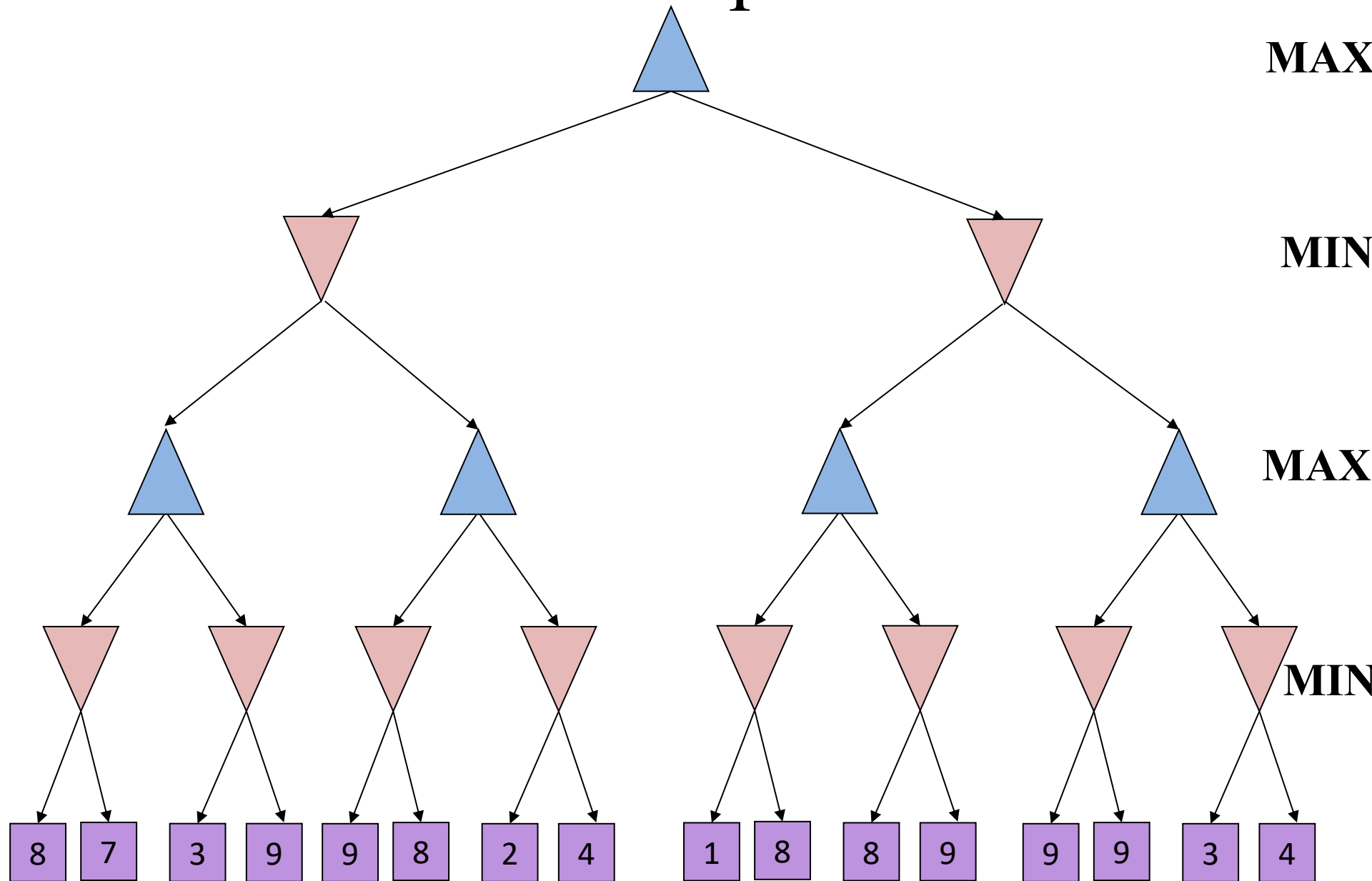
Exemplu



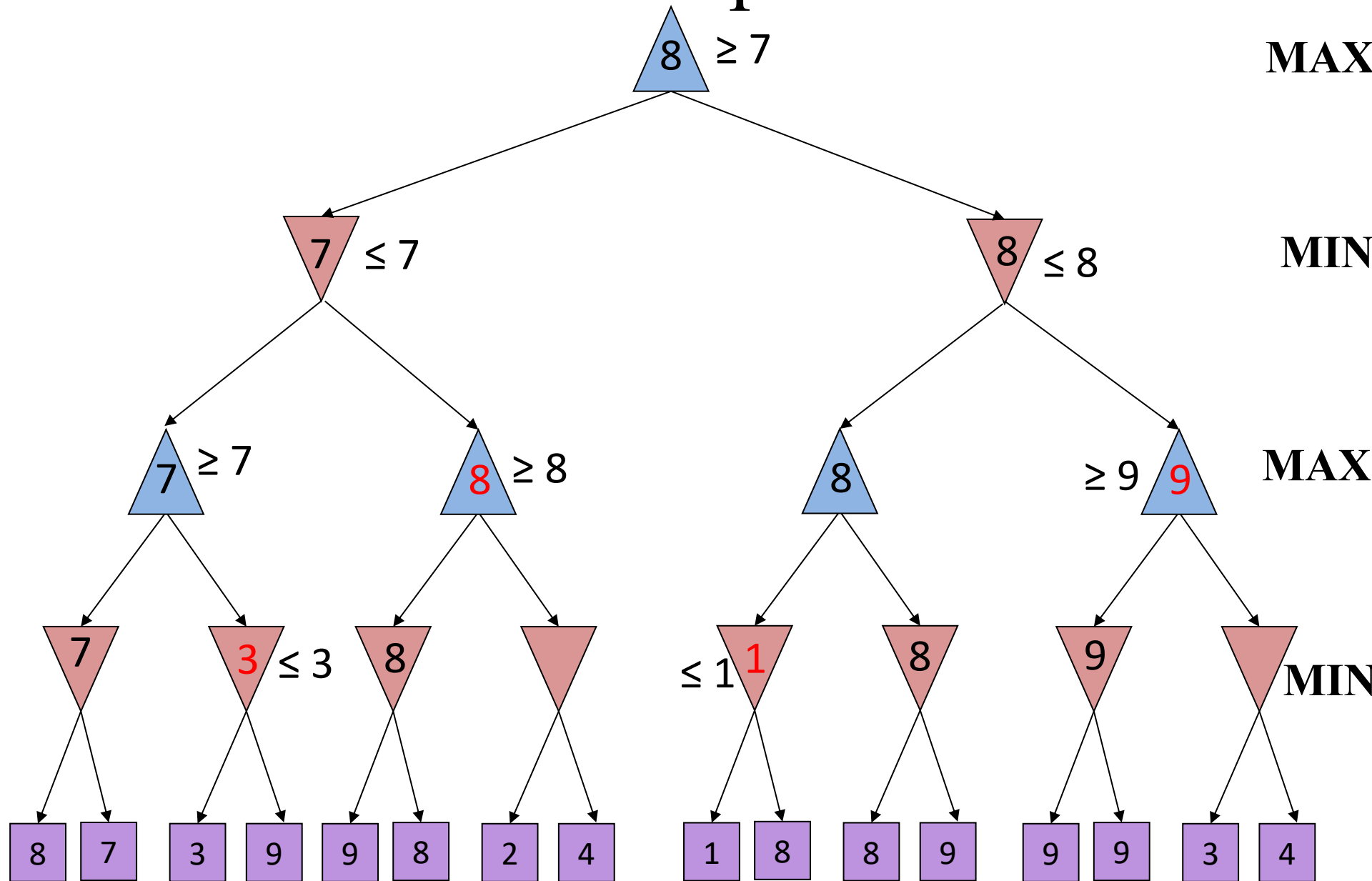
Exemplu



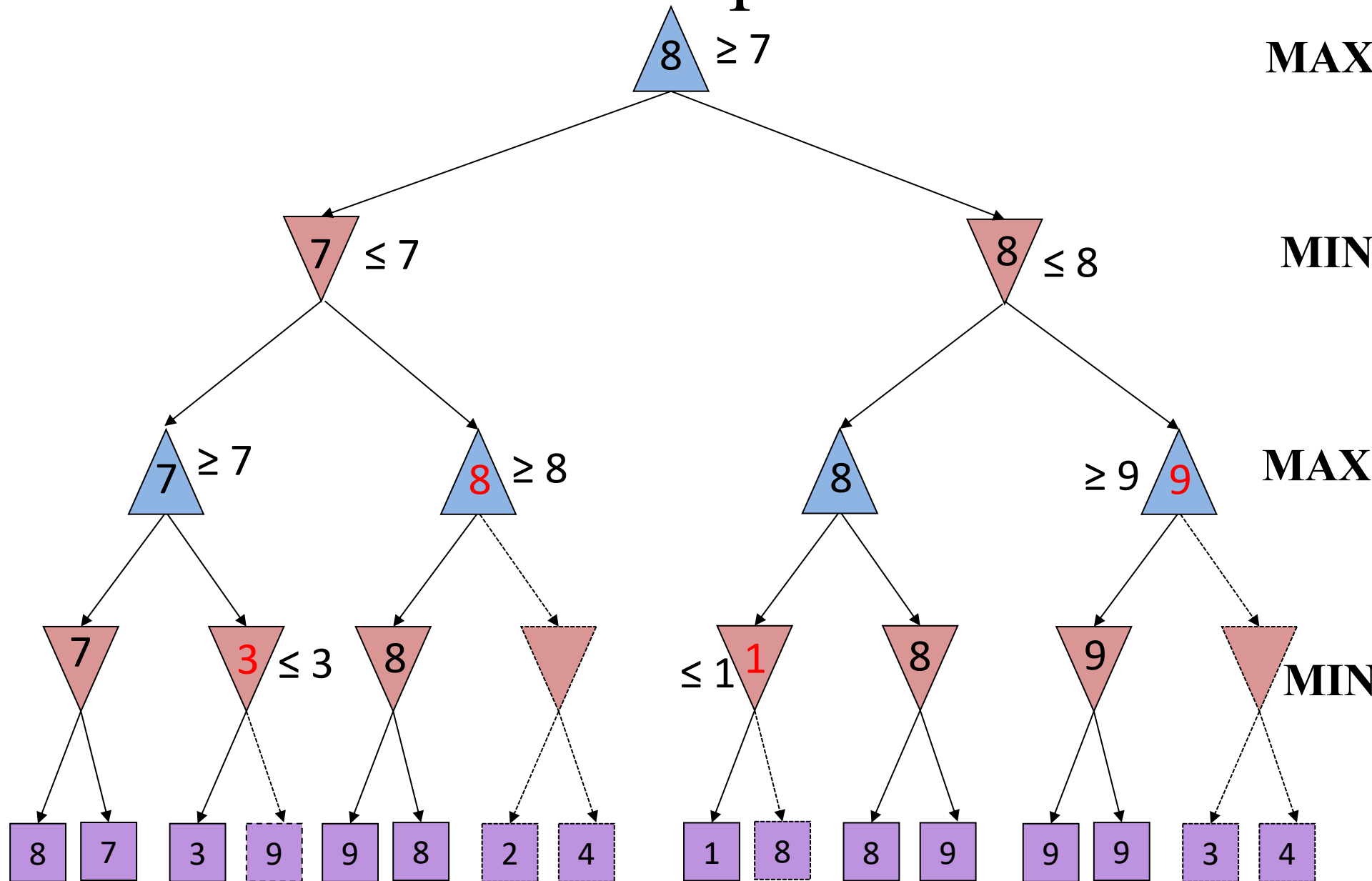
Exemplu



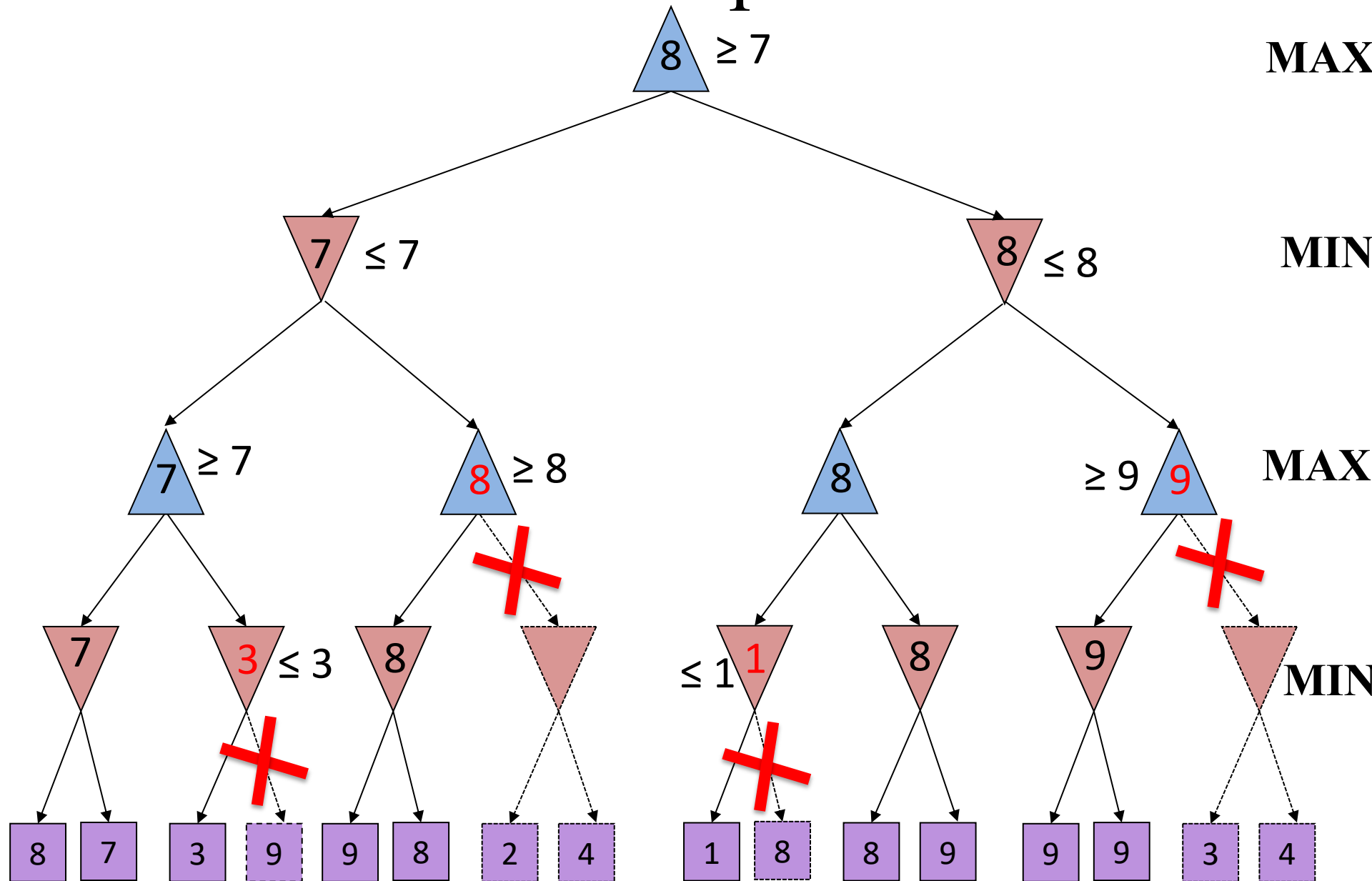
Exemplu



Exemplu

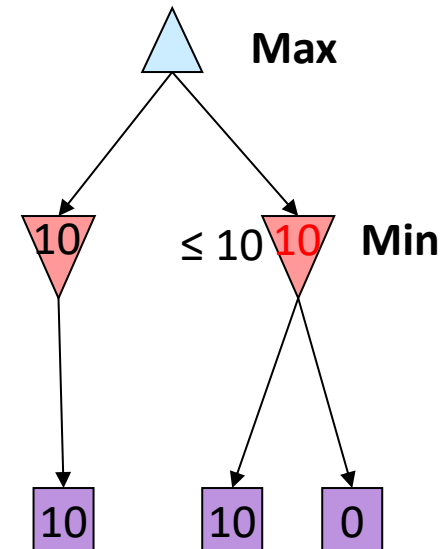


Exemplu

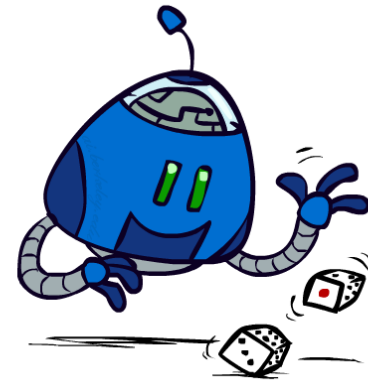
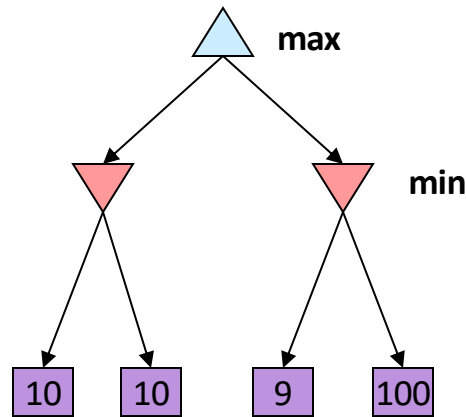
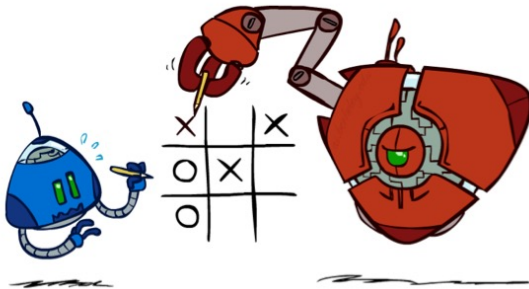


Proprietățile algoritmului Alpha-Beta Pruning

- procesul de retezare nu are niciun efect asupra valorii minimax calculate pentru nodul rădăcină (aceasta nu se schimbă)
- valorile nodurilor intermediare (aproximate) pot fi greșite:
 - important: nodurile fii ale rădăcinii pot avea valori greșiteImplementarea naivă poate conduce la erori
- vizitarea în ordinea bună a nodurilor fii conduce la retezări mai rapide
- cu “ordonare perfectă”:
 - complexitatea timp poate scădea la $O(b^{m/2})$
 - algoritmul poate rula pe o adâncime dublă
 - căutarea exhaustivă nu este totuși posibilă...

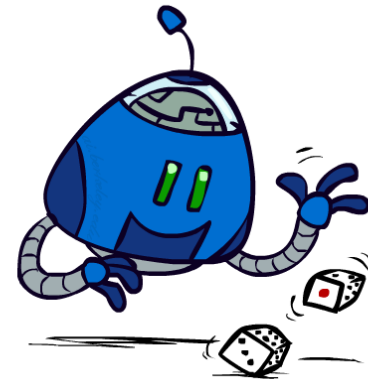
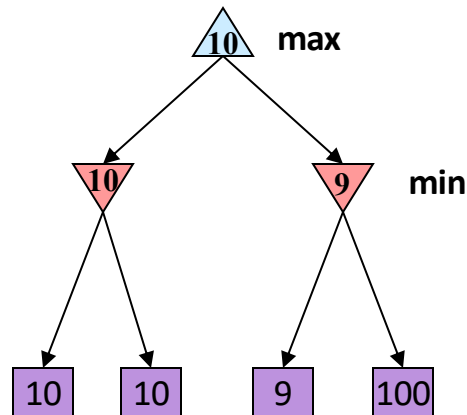
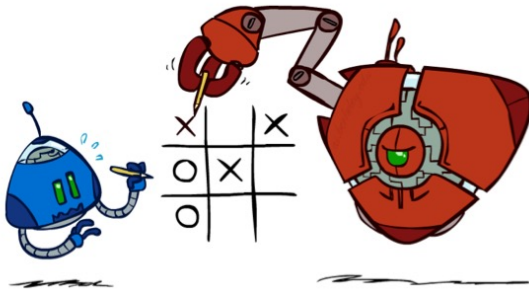


Cazul cel mai defavorabil vs cazul mediu



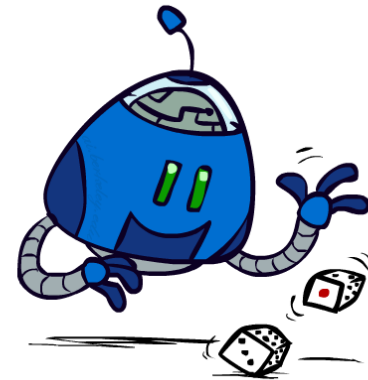
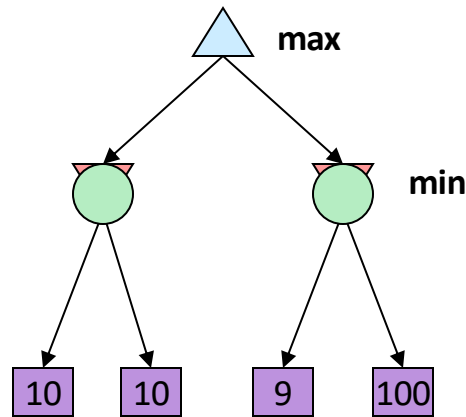
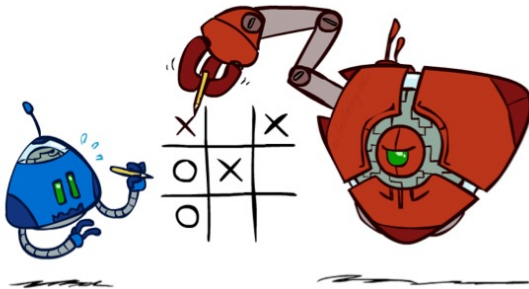
Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Cazul cel mai defavorabil vs cazul mediu



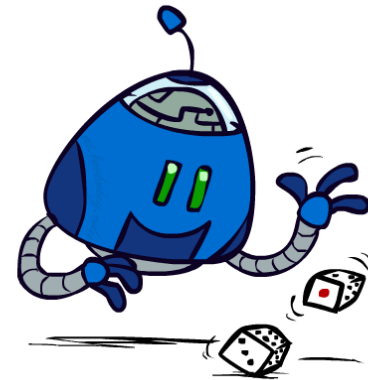
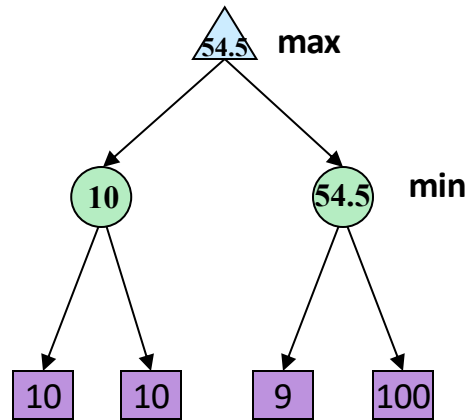
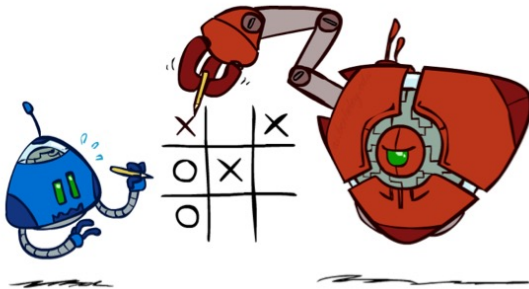
Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Cazul cel mai defavorabil vs cazul mediu



Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

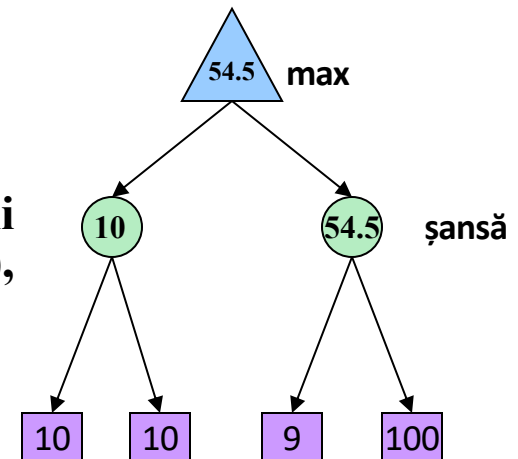
Cazul cel mai defavorabil vs cazul mediu



Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Căutare ExpectiMax

- **De ce anumite acțiuni pot avea un rezultat imprevizibil:**
 - incertitudine în formă explicită: aruncarea unui zar
 - adversari impredictibili: pot avea un comportament aleator
 - anumite acțiuni pot eșua: un robot care execută anumite acțiuni poate avea roțile care alunecă
- **Valorile stărilor ar trebui să reflecte în aceste situații rezultatele/utilitățile/scorul pe cazul mediu (Expectation), nu pe cazul cel mai defavorabil (Min)**
- **Căutare ExpectiMax:** calculează scorul mediu pe baza unei strategii optime a jucătorului Max
 - noduri de tip max la fel ca în căutarea MiniMax
 - noduri de tip șansă sunt un fel de noduri de tip Min dar rezultatul/utilitatea/scorul este incert, fiind guvernat de o distribuție de probabilitate
 - calculează **utilitățile medii** (media ponderată a nodurilor fii) – expectation (medie)
 - în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii



Implementare ExpectiMax

def value(stare):

 dacă stare este o stare terminală: returnează utilitatea stării

 dacă următorul agent care mută este MAX:
 returnează max-value(stare)

 dacă următorul agent care mută este EXP: returnează exp-value(stare)

def max-value(stare):

 initializează $v = -\infty$

 pentru fiecare succesor al stării:

$v = \max(v, \text{value(succesor)})$

 returnează v

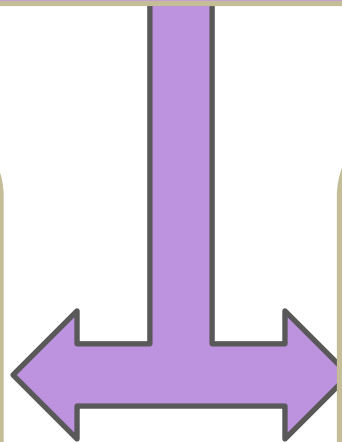
def exp-value(state):

 initializează $v = 0$

 pentru fiecare succesor al stării:

$v = v + p(\text{succesor}) * \text{value(succesor)}$

 returnează v



Implementare ExpectiMax

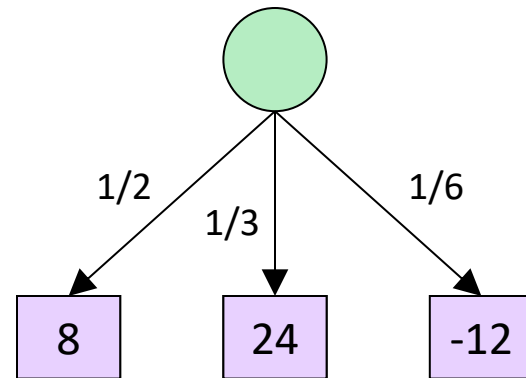
def exp-value(state):

 initializează $v = 0$

 pentru fiecare succesor al stării:

$v = v + p(\text{succesor}) * \text{value}(\text{succesor})$

 returnează v



$$v = (1/2) (8) + (1/3) (24) + (1/6) (-12) = 10$$

Implementare ExpectiMax

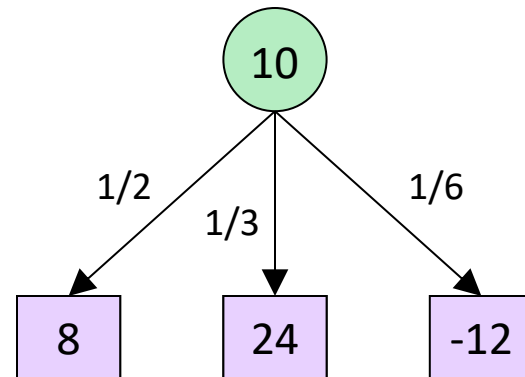
def exp-value(state):

 initializează $v = 0$

 pentru fiecare succesor al stării:

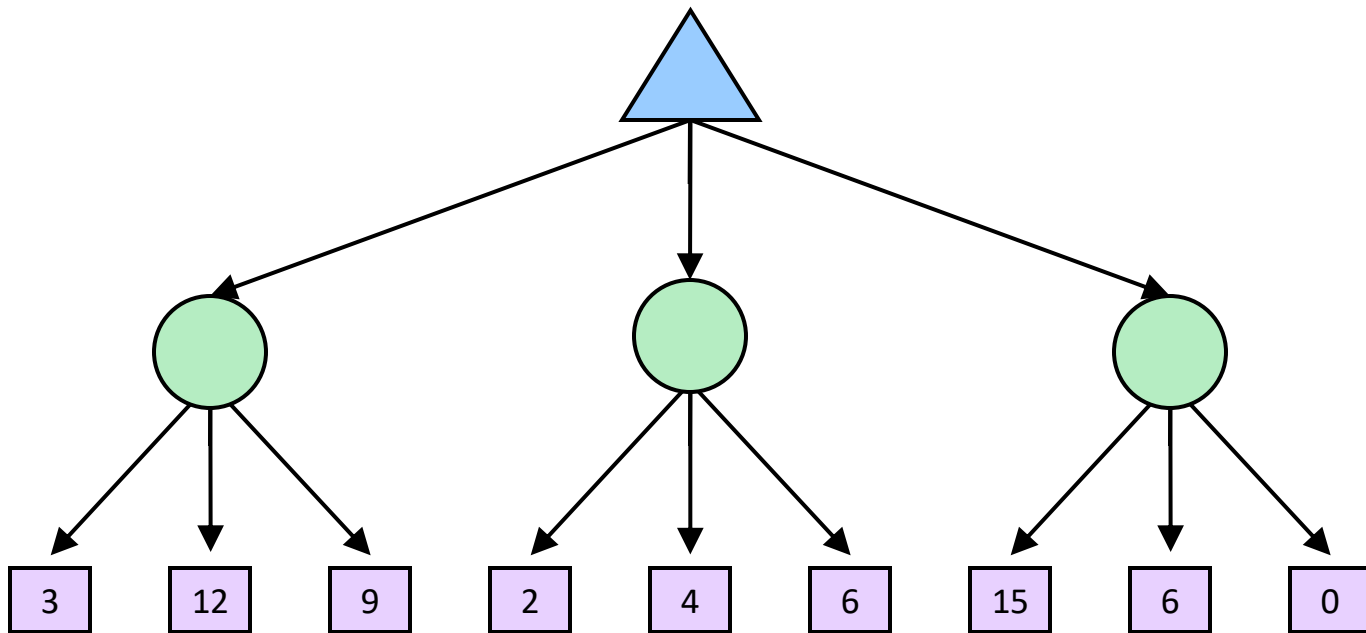
$v = v + p(\text{succesor}) * \text{value}(\text{succesor})$

 returnează v

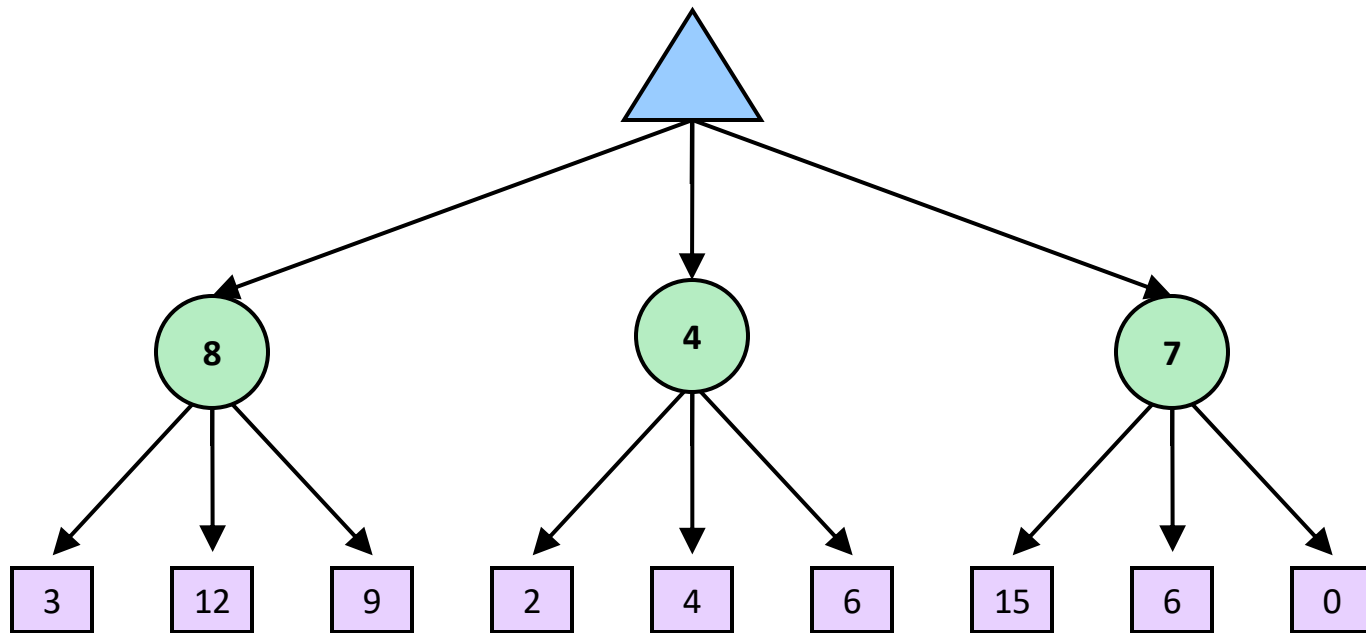


$$v = (1/2) (8) + (1/3) (24) + (1/6) (-12) = 10$$

Exemplu ExpectiMax



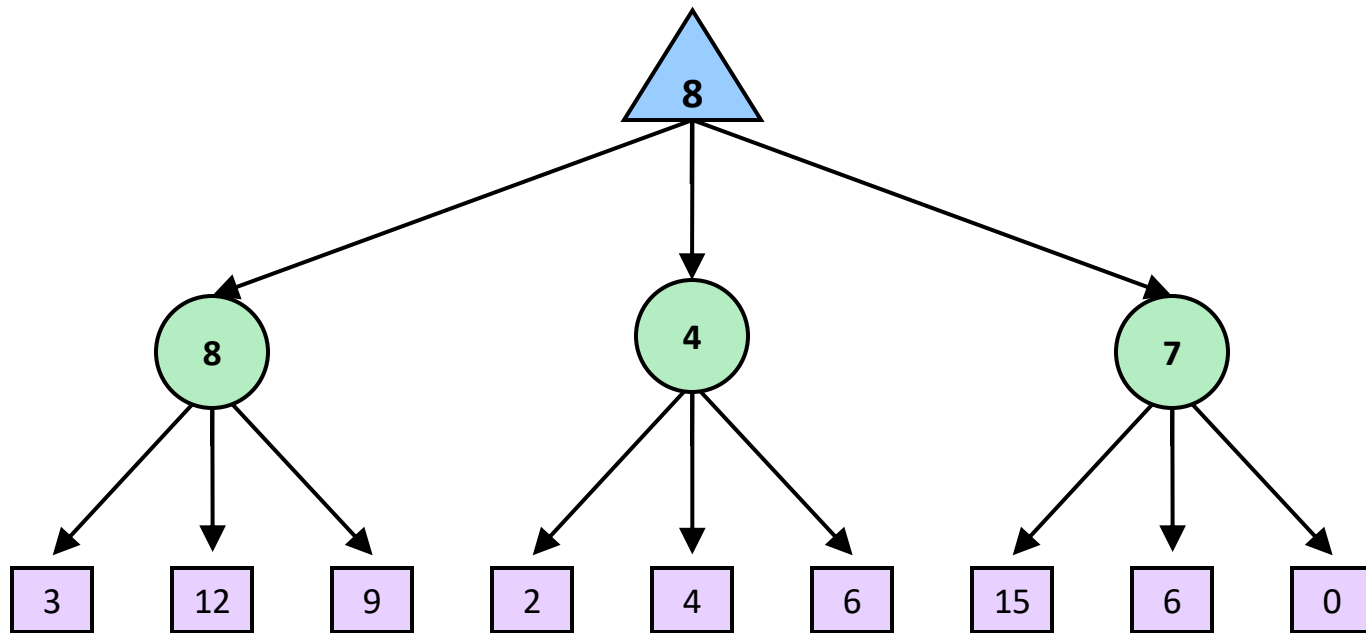
Exemplu ExpectiMax



calculează **utilitățile medii** (media ponderată a nodurilor fii) – expectation (medie)

în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii

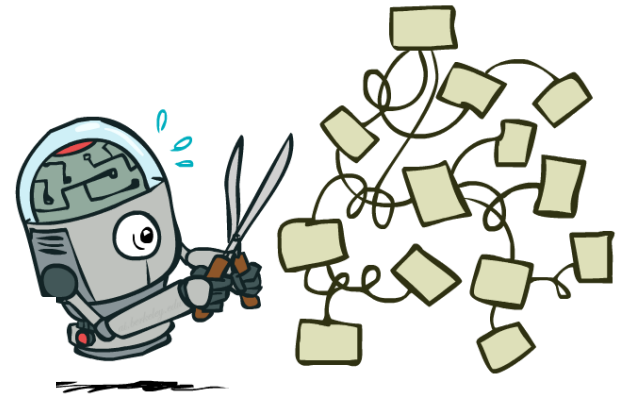
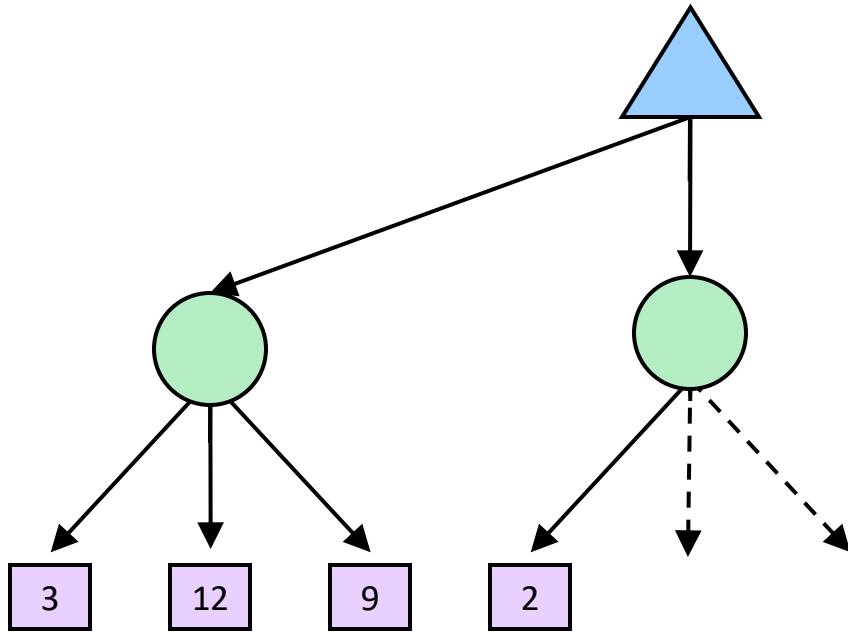
Exemplu ExpectiMax



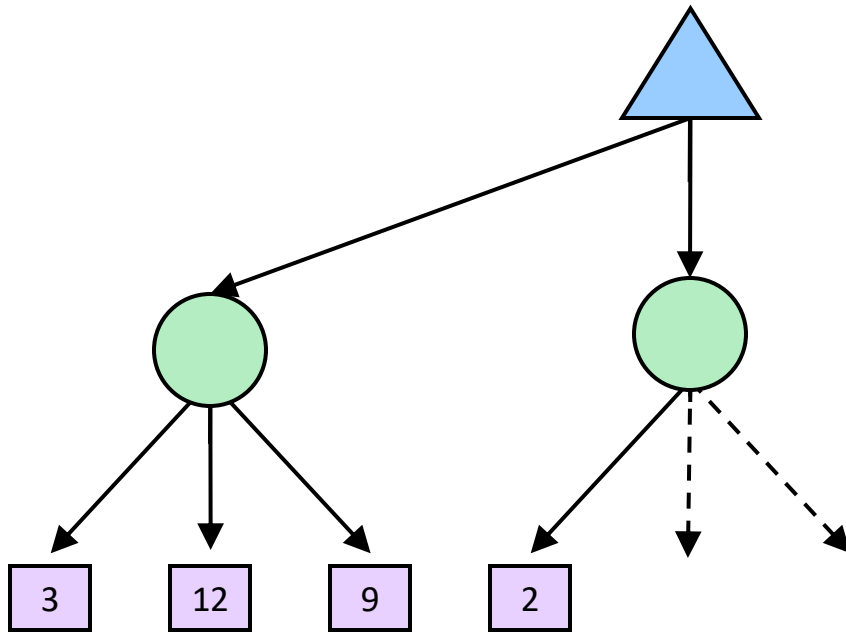
calculează **utilitățile medii** (media ponderată a nodurilor fii) – expectation (medie)

în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii

Accelerarea algoritmului ExpectiMax?

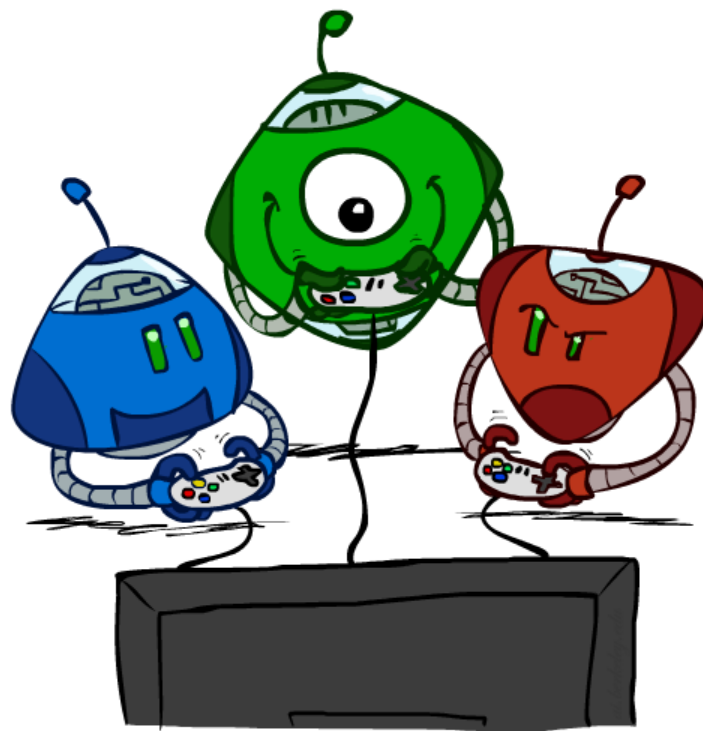


Accelerarea algoritmului ExpectiMax?



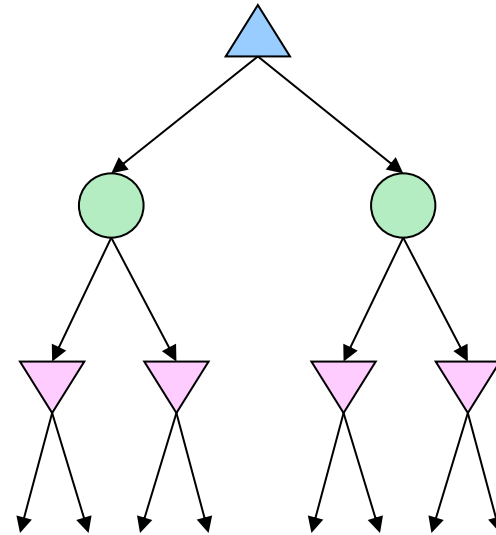
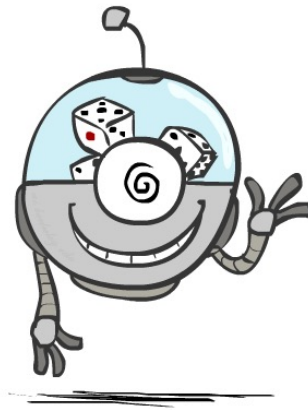
Diferit de algoritmul MiniMax care poate fi accelerat folosind alpha-beta retezare, nu putem accelera algoritmul ExpectiMax întrucât pot exista noduri la dreapta cu o valoare foarte mare care pot influența utilitatea medie.

Alte tipuri de jocuri



Jocuri cu noduri mixte

- E.g. Backgammon (table)
- ExpectiMiniMax
 - mediul (environment) este un “agent aleator” care mută (se dă cu zarul) după fiecare agent Min/Max
 - fiecare nod calculează utilitatea medie a fiilor



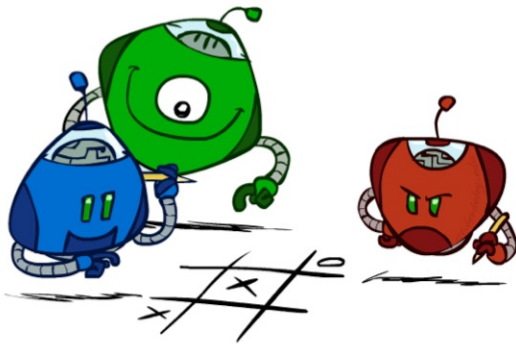
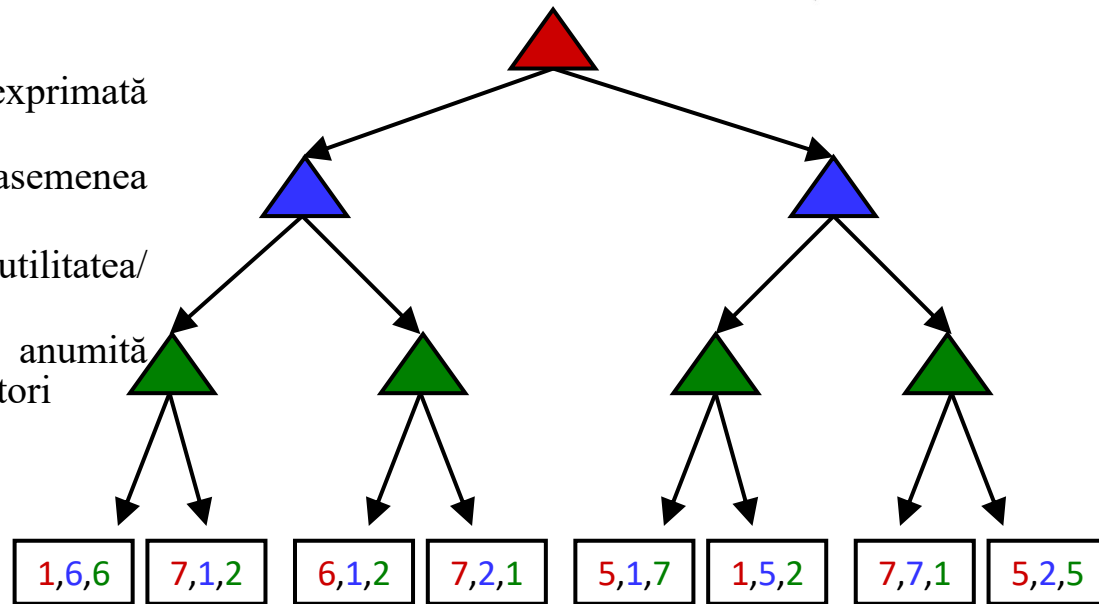
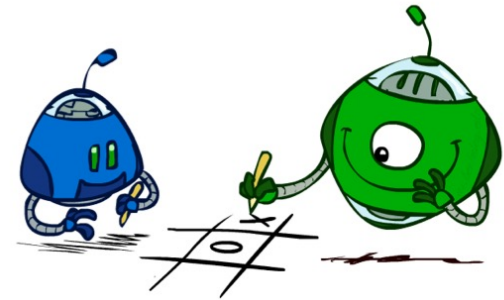
Exemplu: Backgammon (table)

- Aruncările cu zarul cresc factorul de ramificare b : 21 de posibilități cu 2 zaruri (1-1, 1-2, ..., 6-6)
 - Backgammon/Table ≈ 20 mutări legale
 - factor total de ramificare: $b \approx 21 \times 20 \approx 400$
 - pentru o căutare de d mutări, număr de stări = $(21 \times 20)^d$
- Pe măsură ce adâncimea crește, probabilitatea de a ajunge într-un anumit nod se micșorează
 - utilitatea unei căutări se diminuează
 - căutare limitată în adâncime nu pare așa de rea
 - accelerarea unei asemenea abordări este greu de realizat
- Soluția în practică (TD-Gammon)
 - căutare superficială (adâncime mică)
 - funcție de evaluare puternică
 - reinforcement learning
 - performanță: nivel de campion mondial



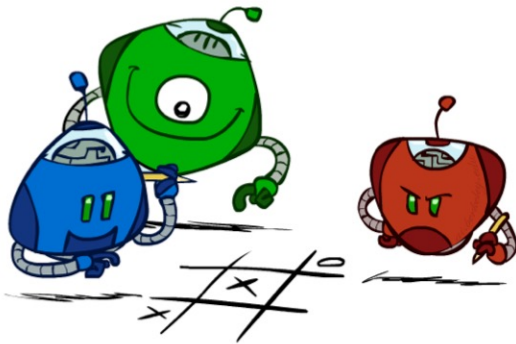
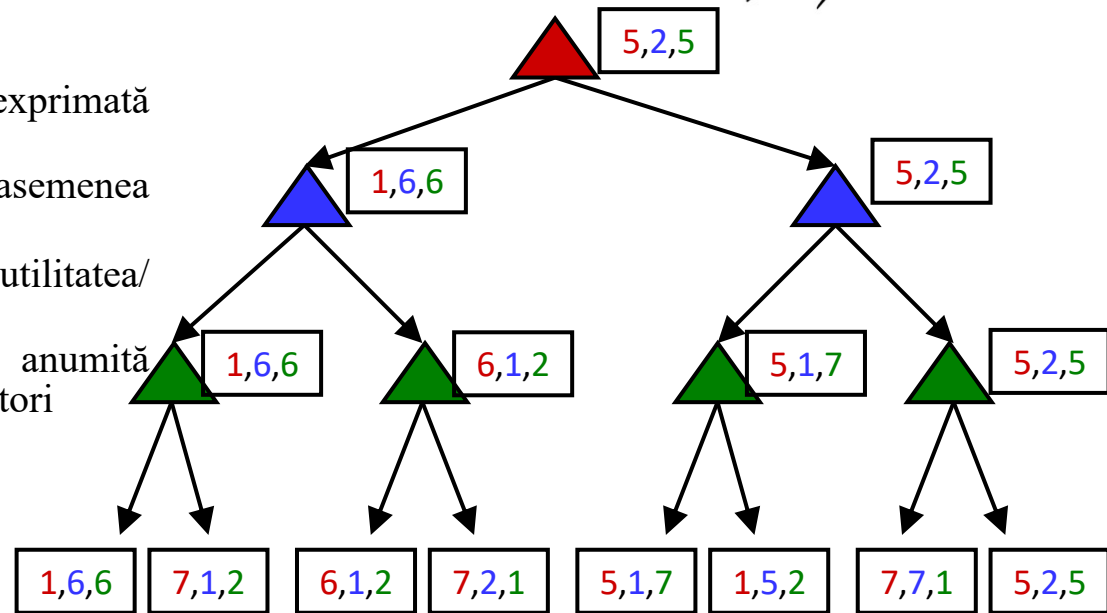
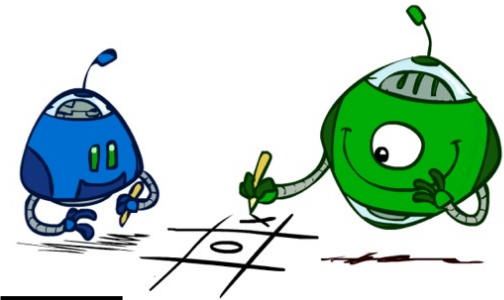
Utilități multi-agent

- Dacă jocul nu este cu sumă zero sau are mai mult de doi jucători?
- Generalizare pentru MiniMax:
 - nodurile terminale au utilitatea exprimată printr-un tuplu
 - valorile nodurilor interne sunt de asemenea tupluri de utilitate
 - fiecare jucător maximizează utilitatea/ componenta proprie
 - poate apărea în mod dinamic o anumită cooperare sau competiție între jucători



Utilități multi-agent

- Dacă jocul nu este cu sumă zero sau are mai mult de doi jucători?
- Generalizare pentru MiniMax:
 - nodurile terminale au utilitatea exprimată printr-un tuplu
 - valorile nodurilor interne sunt de asemenea tupluri de utilitate
 - fiecare jucător maximizează utilitatea/ componenta proprie
 - poate apărea în mod dinamic o anumită cooperare sau competiție între jucători



Probleme de satisfacere de constrângeri

- probleme standard de căutare:
 - spațiul stărilor, stare inițială, stare finală, acțiuni, costul unei acțiuni
 - test dacă o stare este stare-scop poate fi orice funcție de stări
 - funcția succesor poate fi orice
- căutarea unei soluții: secvență de acțiuni
 - drumul până la nodul scop este important
 - drumurile au costuri diferite, au adâncimi diferite
 - euristicile specifice problemelor ajută la găsirea soluției
- de cele mai multe ori putem folosi o reprezentare pentru fiecare stare sub forma unor variabile, fiecare luând anumite valori
 - soluția se obține când toate variabilele au asignată o valoare care respectă toate constrângerile problemei
 - drumurile au aceeași adâncime (același număr de variabile de asignat)
 - obținem probleme de satisfacere a constrângerilor (PSC)

Probleme de satisfacere a constrângerilor -definiție-

O problemă de satisfacere a constrângerilor (PSC) are **trei componente**:

- X = o **mulțime de variabile**, $X = \{X_1, X_2, \dots, X_n\}$
- D = o **mulțime de domenii**, $D = \{D_1, D_2, \dots, D_n\}$, unde fiecare domeniu $D_i = \{v_{i1}, v_{i2}, \dots, v_{ik_i}\}$ corespunde valorilor posibile ale variabilei X_i
- C = o **mulțime de constrângeri**, $C = \{C_1, C_2, \dots, C_n\}$, unde fiecare constrângere C_i definește o relație pe care valorile anumitor variabile trebuie să o îndeplinească
 - constrângerile pot fi exprimate implicit sau explicit

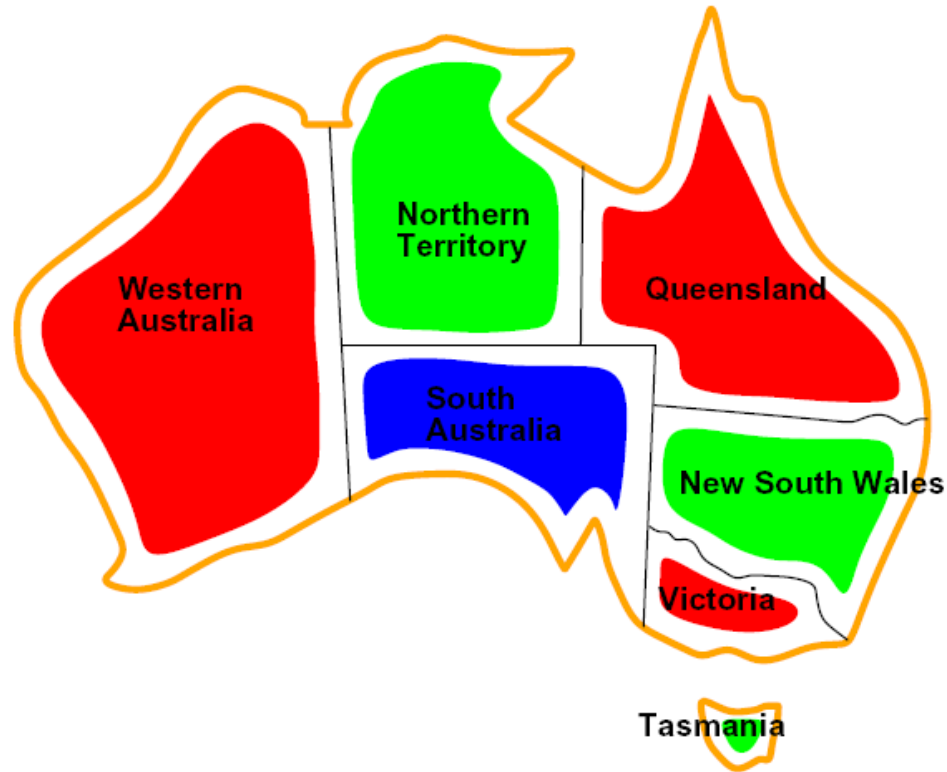
Exemplu – problema colorării unei hărți

Problema de a colora o harta cu un număr de 3 culori (**roșu**, **verde**, **albastru**) astfel încât regiunile adiacente să aibă culori diferite



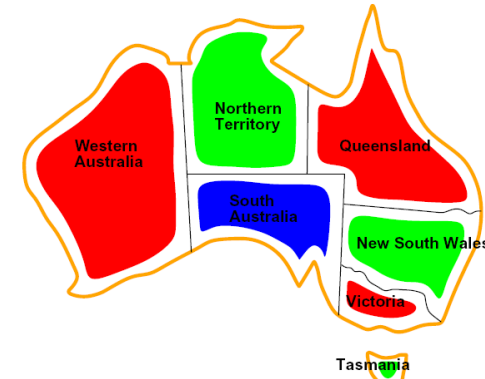
Exemplu – problema colorării unei hărți

O posibilă soluție (există mai multe)



Exemplu – problema colorării unei hărți

- Variabile: $X = \{WA, NT, Q, NSW, V, SA, T\}$
- Domenii: $D = D_i = \{\text{roșu}, \text{verde}, \text{albastru}\}$
- Constrângeri: regiuni adiacente trebuie să aibă culori diferite



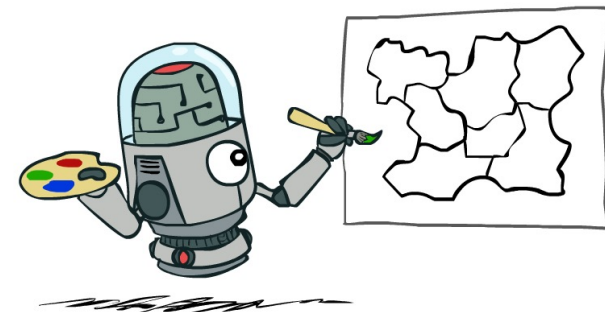
$C = \{C_1, C_2, C_3, C_4, C_5, C_6, C_7, C_8, C_9\}$

C_1 sub formă implicită: $WA \neq NT$

C_1 sub formă explicită: $(WA, NT) \in \{(\text{roșu}, \text{verde}), (\text{roșu}, \text{albastru}), (\text{verde}, \text{albastru}), (\text{verde}, \text{roșu}), (\text{albastru}, \text{roșu}), (\text{albastru}, \text{verde})\}$

- Soluțiile sunt reprezentate de asignarea unor valori variabilelor care satisfac toate constrângerile:

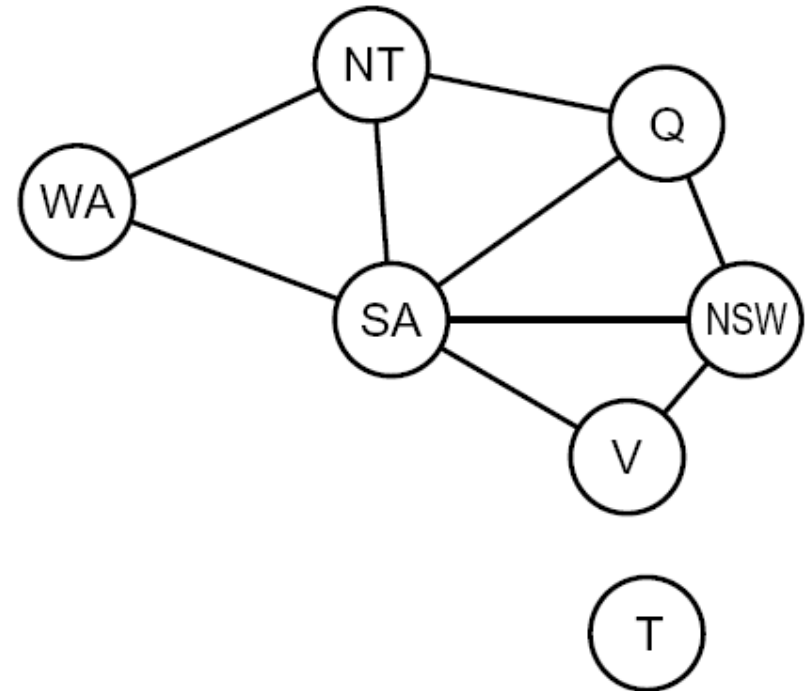
$\{WA = \text{roșu}, NT = \text{verde}, Q = \text{roșu}, NSW = \text{verde}, V = \text{roșu}, SA = \text{albastru}, T = \text{verde}\}$



Vizualizarea unei PSC sub forma unui graf de constrângeri



Problemă inițială

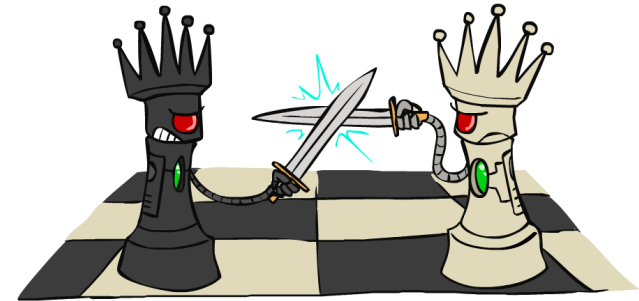
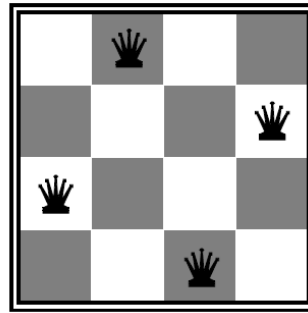


Problemă inițială reprezentată sub forma unui graf de constrângeri

Exemplu – problema celor N regine

- Formulare 1:

- variabile: X_{ij}
- domenii: $\{0, 1\}$
- constrângeri:



$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$$

Piese de pe aceeași linie

$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$$

Piese de pe aceeași coloană

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

Piese de pe aceeași diagonală

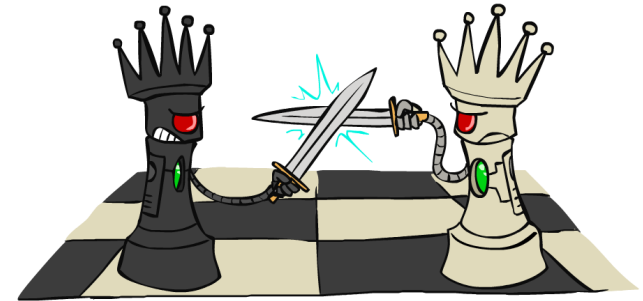
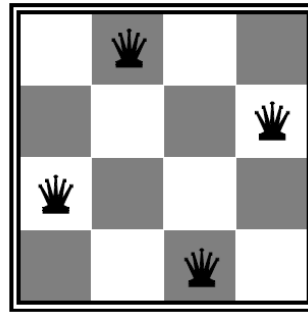
$$\forall i, j, k \quad (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

- pe baza acestor constrângeri puteți să găsiți o soluție?

- soluția banală $X_{ij} = 0$

Exemplu – problema celor N regine

- Formulare 1:
 - variabile: X_{ij}
 - domenii: $\{0, 1\}$
 - constrângeri:



$$\forall i, j, k \quad (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$$

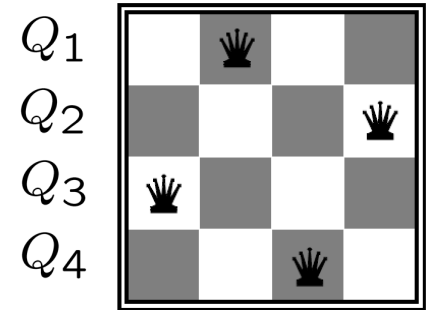
$$\forall i, j, k \quad (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\forall i, j, k \quad (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$$

$$\sum_{i,j} X_{ij} = N$$

Exemplu – problema celor N regine

- Formulare 2:
 - variabile: Q_k
 - domenii (coloane): $\{1, 2, 3, \dots, N\}$
 - constrângeri:
 - implicite: orice două regine Q_i și Q_j nu se atacă ($i \neq j$)
 - explicite: $(Q_1, Q_2) \in \{(1,3), (1,4), (2,4), (3,1), (4,1), (4,2)\}$
 - ...



Exemplu – Criptaritmetica

$$\begin{array}{r} \text{T W O} \\ + \text{T W O} \\ \hline \text{F O U R} \end{array}$$

Exemplu – Criptaritmetica

- Variabile:

$F \ T \ U \ W \ R \ O \ X_1 \ X_2 \ X_3$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$

- Domenii:

$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

- Constrângeri:

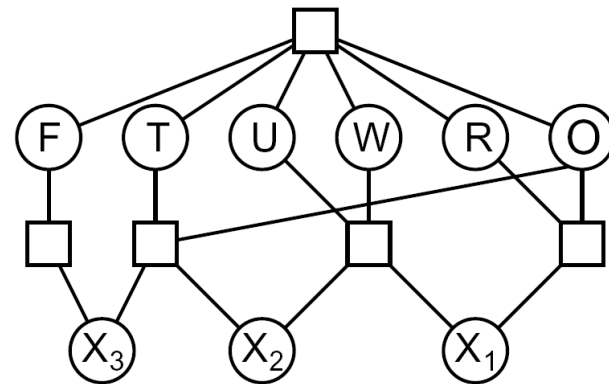
$\text{alldiff}(F, T, U, W, R, O)$

$$O + O = R + 10 \cdot X_1$$

$$W + W + X_1 = U + 10 \cdot X_2$$

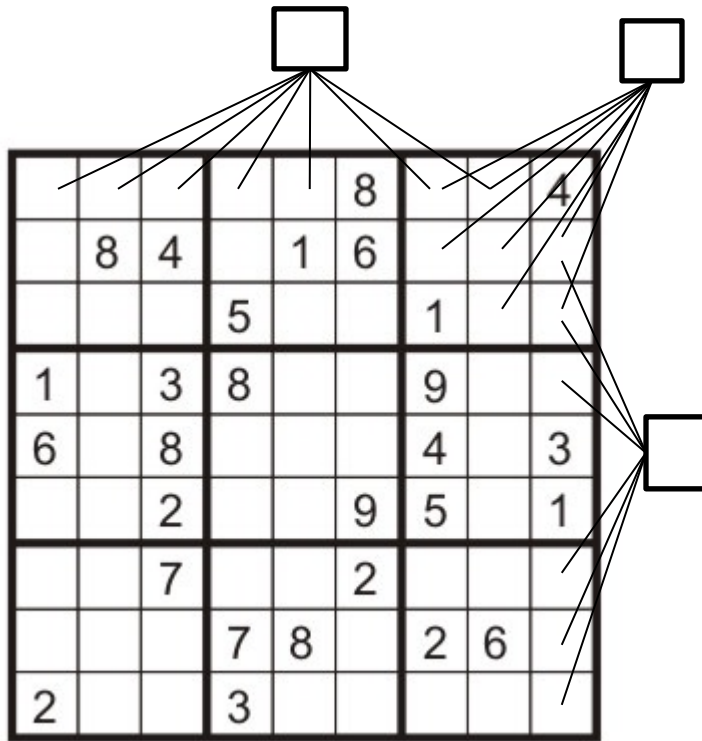
$$T + T + X_2 = O + 10 \cdot X_3$$

$$F = X_3$$



Problemă inițială reprezentată sub forma unui hiper-graf de constrângeri
Apar constrângeri de aritate 2, 3, 4, 6.

Exemplu – Sudoku



- Variabile:

- fiecare pătrat gol

- Domenii:

- $\{1, 2, \dots, 9\}$

- Constrângeri:

Pe fiecare coloană sunt 9 cifre diferite

Pe fiecare linie sunt 9 cifre diferite

În fiecare pătrat 3×3 sunt 9 cifre distincte
(sau putem adăuga multe inegalități de tipul $A1 \neq A2$, $A1 \neq A3$, etc.)

Zebra puzzle

6.7 Consider the following logic puzzle: In five houses, each with a different color, live five persons of different nationalities, each of whom prefers a different brand of candy, a different drink, and a different pet. Given the following facts, the questions to answer are “Where does the zebra live, and in which house do they drink water?”

The Englishman lives in the red house.

The Spaniard owns the dog.

The Norwegian lives in the first house on the left.

The green house is immediately to the right of the ivory house.

The man who eats Hershey bars lives in the house next to the man with the fox.

Kit Kats are eaten in the yellow house.

The Norwegian lives next to the blue house.

The Smarties eater owns snails.

The Snickers eater drinks orange juice.

The Ukrainian drinks tea.

The Japanese eats Milky Ways.

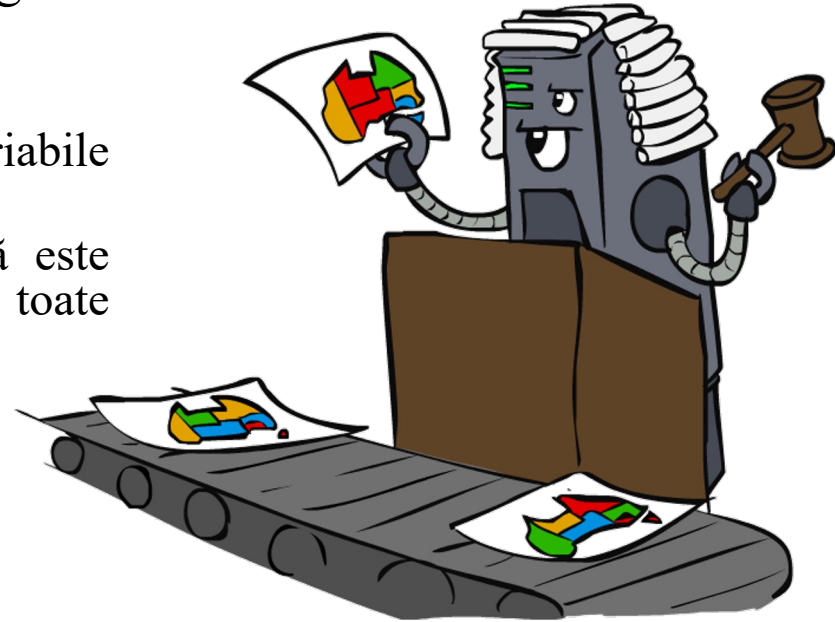
Kit Kats are eaten in a house next to the house where the horse is kept.

Coffee is drunk in the green house.

Milk is drunk in the middle house.

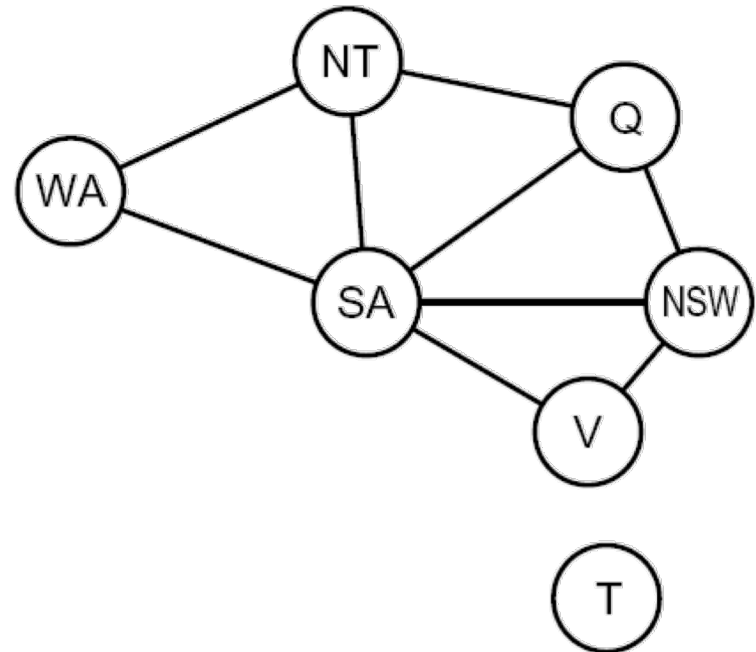
Formulara standard a unei PSC pentru căutare

- stările sunt definite de valorile asignate până la momentul curent (fac parte din asignări parțiale)
 - stare inițială : asignarea vidă, $\{\}$
 - funcția succesori: asignez o valoare unei variabile neasignate
 - testul de stare-scop: dacă asignarea curentă este completă și consistentă (satisface toate constrângerile)
- pornim cu o soluție simplă, naivă pe care o vom îmbunătăți



Strategii de căutare

- Care este rezultatul căutării în lățime?

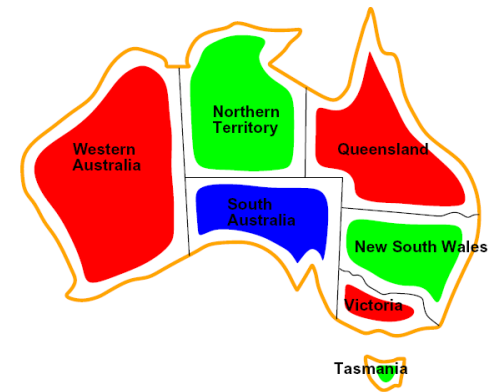


Strategii de căutare

- Care este rezultatul căutării în lăţime?

{ }

{WA=g} {WA=r} ... {NT=g} ...



Recap: Proprietățile căutării în lățime

- Ce fel de noduri expandează BF?

- procesează toate nodurile deasupra soluției de adâncime minimă
- fie s = adâncimea minimă a unui nod scop
- complexitate timp $O(b^s)$
- verifică dacă un nod succesori generat este nod scop - reduce complexitatea timp de la $O(b^{s+1})$ la $O(b^s)$

- Cât de mult spațiu necesită BF?

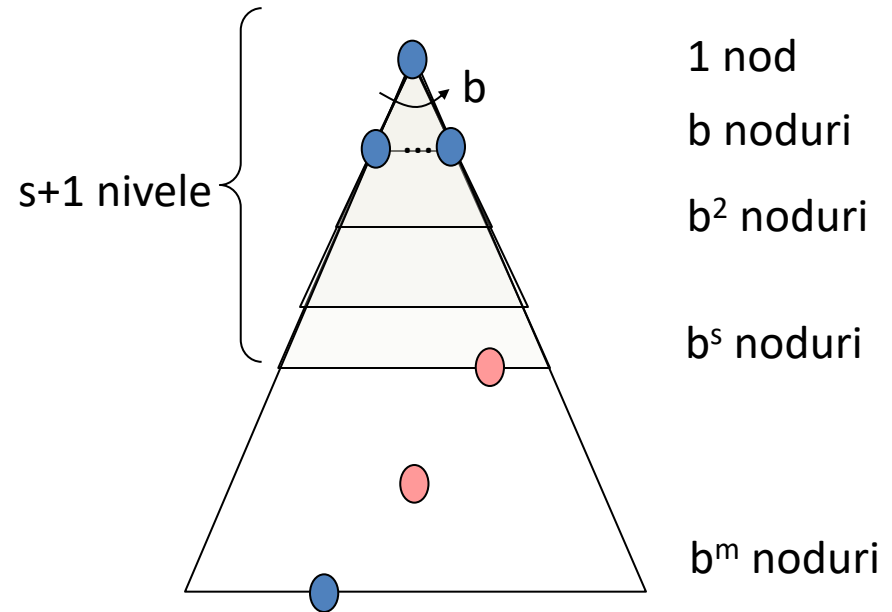
- este spațiul pentru memorarea frontierei $O(b^s)$ + memorarea nodurilor explorate $O(b^{s-1})$
- limită superioară dată de ultimul nivel
- complexitate $O(b^s)$

- Completitudine?

- dacă o soluție există atunci s este finit, deci DA!

- Optimalitate?

- în general NU!
- uneori DA (spre exemplu dacă costurile fiecărei muchii sunt egale)



Strategii de căutare

- Care este rezultatul căutării în lățime?
 - Generează întreg arborele de căutare, soluțiile sunt pe ultimul nivel
 - La nivelul 1, factor de ramificare $b = n \times d$ (avem n variabile și fiecare poate lua d valori)
 - La nivelul 2, factor de ramificare $b = (n-1) \times d$
 - Avem n nivele (= numărul de variabile)
 - Numărul de frunze este $n! \times d^n$, mult mai mare decât d^n

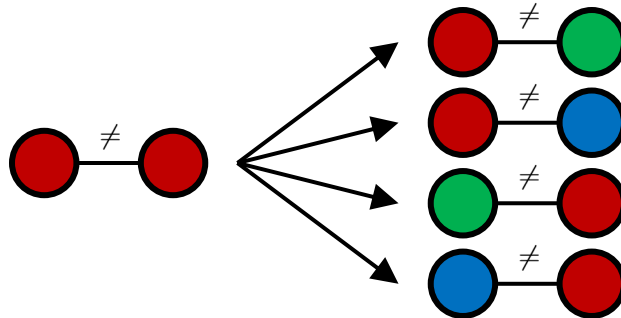
Căutare locală



Căutare locală

- în multe probleme de căutare nu contează drumul către o soluție, ci numai starea finală = soluția (problema celor N regine, problema colorării hărților)
- arborii de căutare păstrează în frontieră alternative neexplorate (asigură completitudinea)
- **căutare locală:** îmbunătățește o stare actuală până când nu mai poți (nu există conceptul de frontieră!)

- funcția succesori: schimbări locale



- în general, o asemenea abordare este mult mai rapidă și mai eficientă din punct de vedere al memoriei (dar este incompletă și suboptimală)

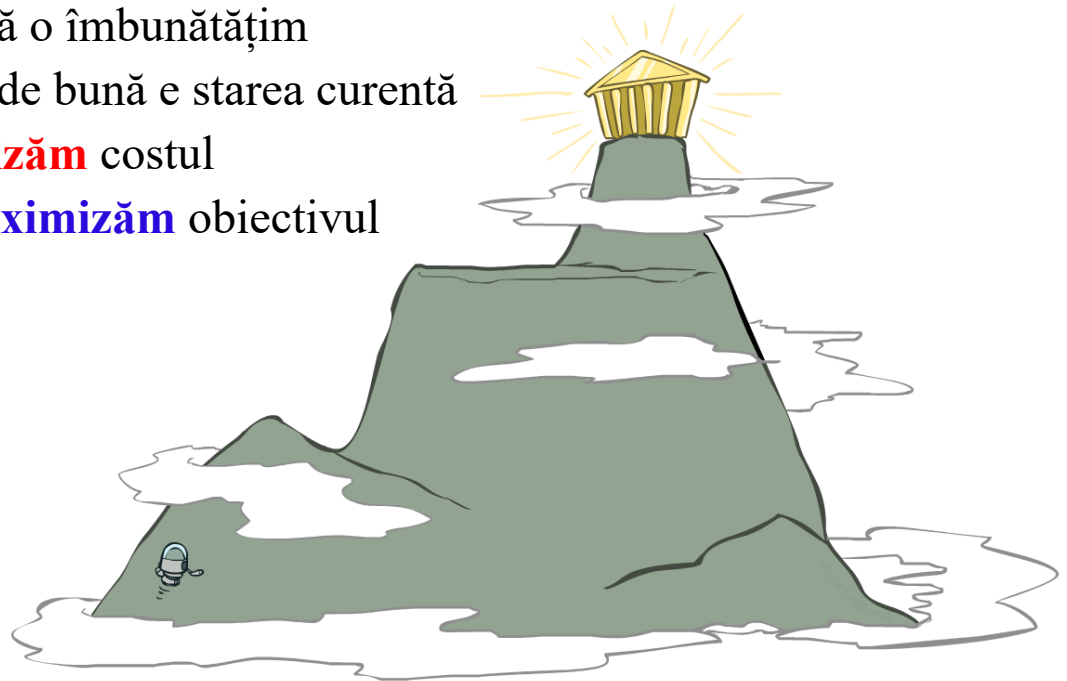
Hill climbing

- Idee generală:
 - pornește de oriunde (stare actuală = asignare aleatoare)
 - repetă: îmbunătățește starea actuală căutând local cea mai bună stare într-o vecinătate
 - dacă nu există vecini mai buni te oprești
- Dezavantaje ale unei asemenea abordări?
 - nu există garanția de completitudine
 - nu există garanția de optimalitate
- Avantaje
 - o putem aplica pentru orice problemă
 - foarte rapid și eficient dpdv al memoriei
- Se mai numește *căutare prin ascensiune*



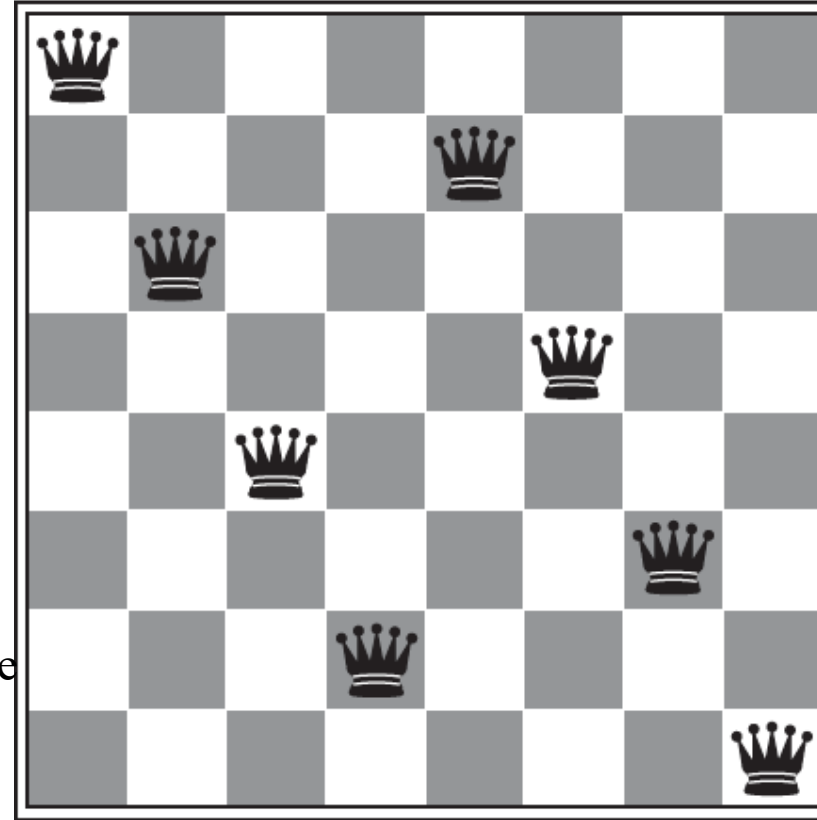
Hill climbing

- Idee generală:
 - pornește de oriunde (stare actuală = asignare aleatoare)
 - repetă: **îmbunătățește starea actuală** căutând local cea mai bună stare într-o vecinătate
 - dacă nu există vecini mai buni te oprești
- Îmbunătățirea unei stări?
 - există o funcție pe care vrem să o îmbunătățim
 - funcția cuantifică numeric cât de bună e starea curentă
 - **funcție cost** – vrem să **minimizăm** costul
 - **funcție obiectiv** – vrem să **maximizăm** obiectivul



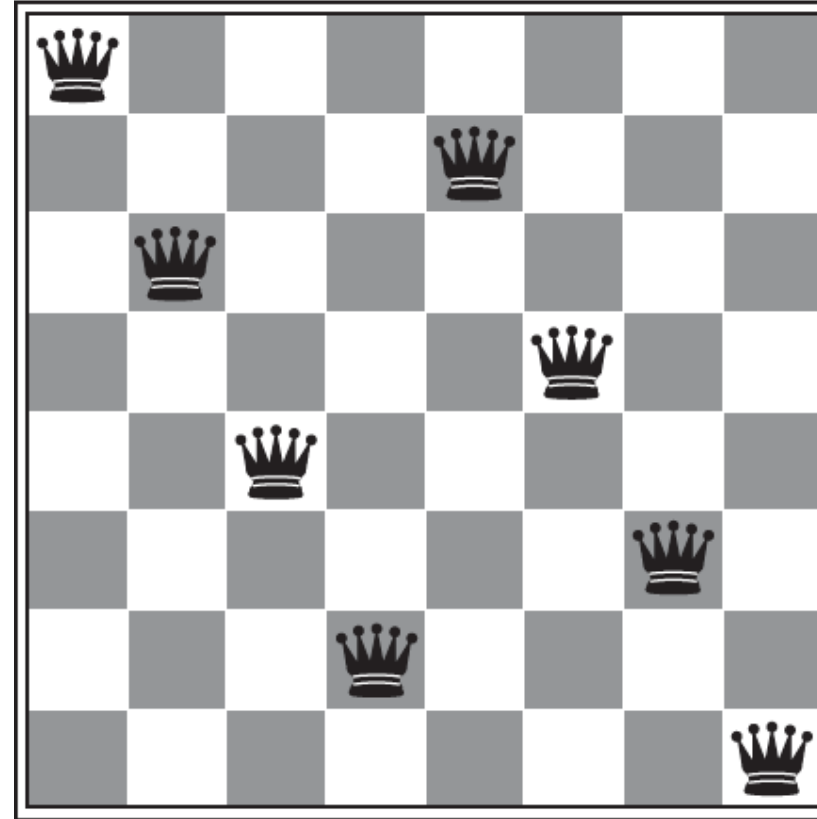
Hill climbing pentru problema celor 8 regine

- **căutare locală:** îmbunătățește o stare actuală până când nu mai poți
- o stare actuală = asignarea pe fiecare din cele 8 coloane a unei regine pe o linie
- funcția succesori (vecinătatea unei stări) = toate stările în care putem ajunge mutând o regină pe o coloană
- sunt 56 de succesori pentru orice stare
- **funcție cost h** = numărul de perechi de regine care se atacă
- vrem să **minimizăm această funcție cost**, minimul global al funcției $h = 0$, corespunde unei soluții (stări finale)



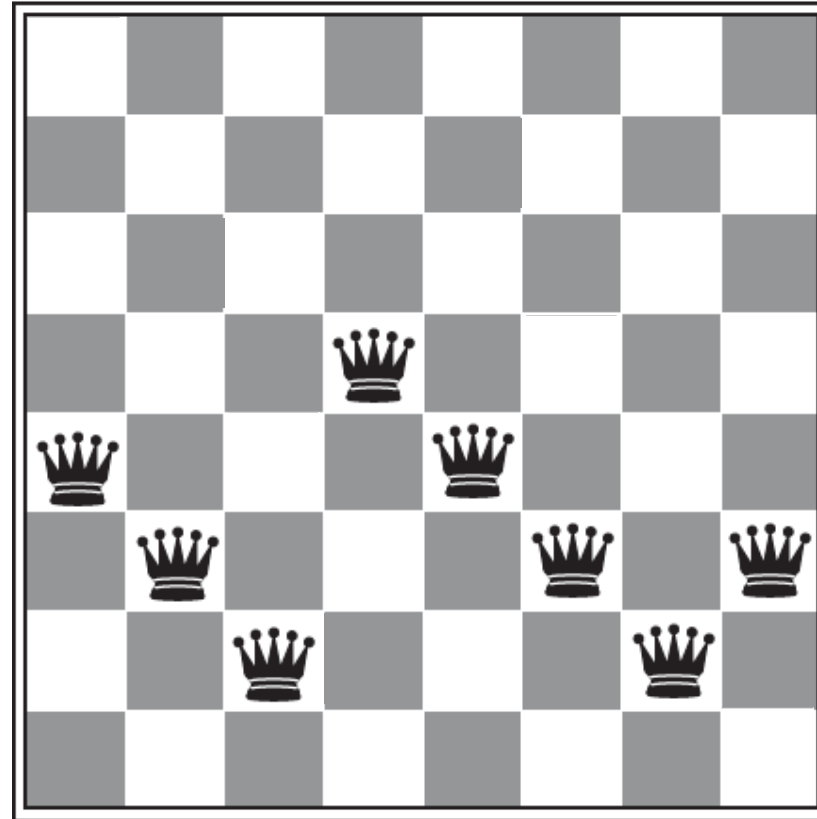
Hill climbing pentru problema celor 8 regine

- **căutare locală:** îmbunătățește o stare actuală până când nu mai poți
- o stare actuală = asignarea pe fiecare din cele 8 coloane a unei regine pe o linie
- funcția succesori (vecinătatea unei stări) = toate stările în care putem ajunge mutând o regină pe o coloană
- sunt 56 de succesori pentru orice stare
- **funcție obiectiv g** = numărul de perechi de regine care NU se atacă
- vrem să **maximizăm această funcție obiectiv**, maximul global al funcției $g = 28$, corespunde unei soluții (stări finale)



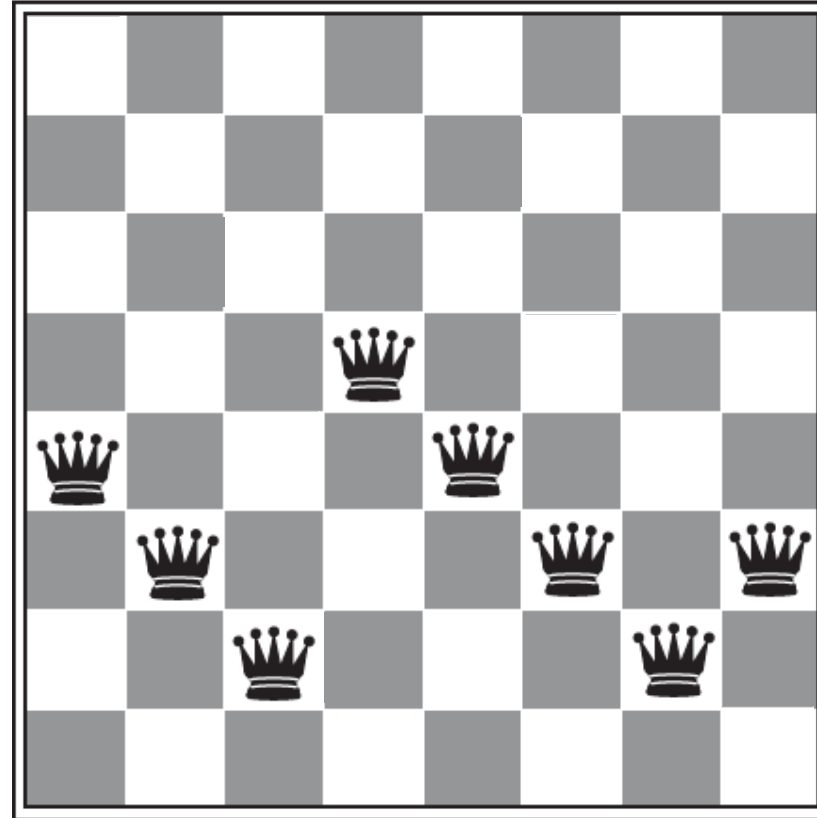
Hill climbing pentru problema celor 8 regine

- **funcție cost h** = numărul de perechi de regine care se atacă
- vrem să **minimizăm această funcție cost**, minimul global al funcției $h = 0$, corespunde unei soluții (stări finale)
- cât este valoarea lui h în starea actuală?
- $h = \text{\#perechi regine pe aceeași coloană} + \text{\#perechi regine pe aceeași linie} + \text{\#perechi regine pe aceeași diagonală}$
- $h = 0 + 5 + 12 = 17$



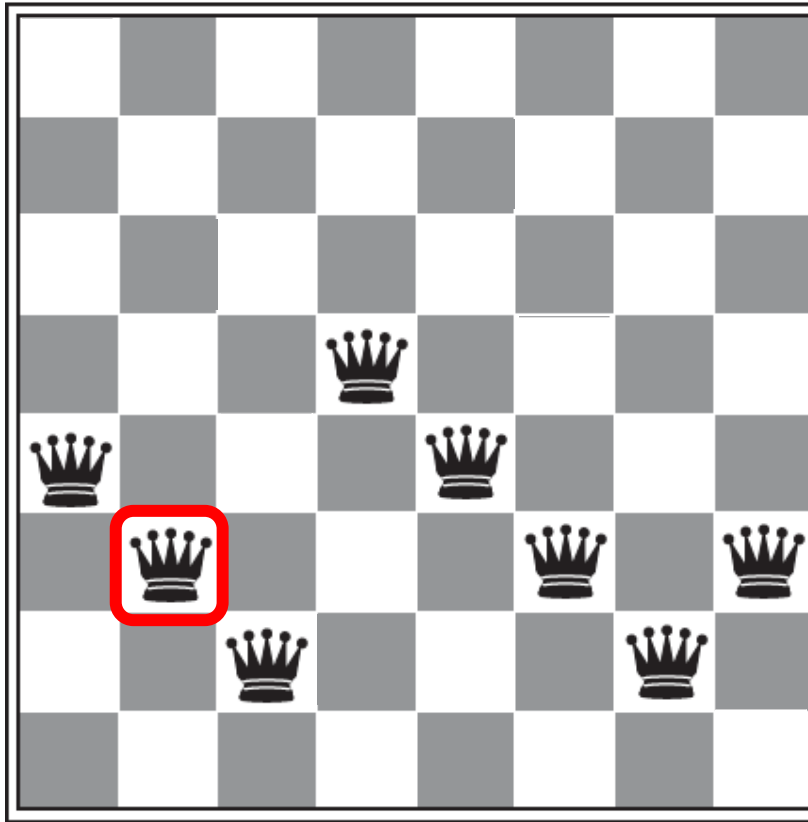
Hill climbing pentru problema celor 8 regine

- **funcție cost h** = numărul de perechi de regine care se atacă
- vrem să **minimizăm această funcție cost**, minimul global al funcției $h = 0$, corespunde unei soluții (stări finale)
- sunt 56 de succesori pentru stare actuală
- calculăm funcția h pentru fiecare succesor

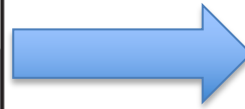
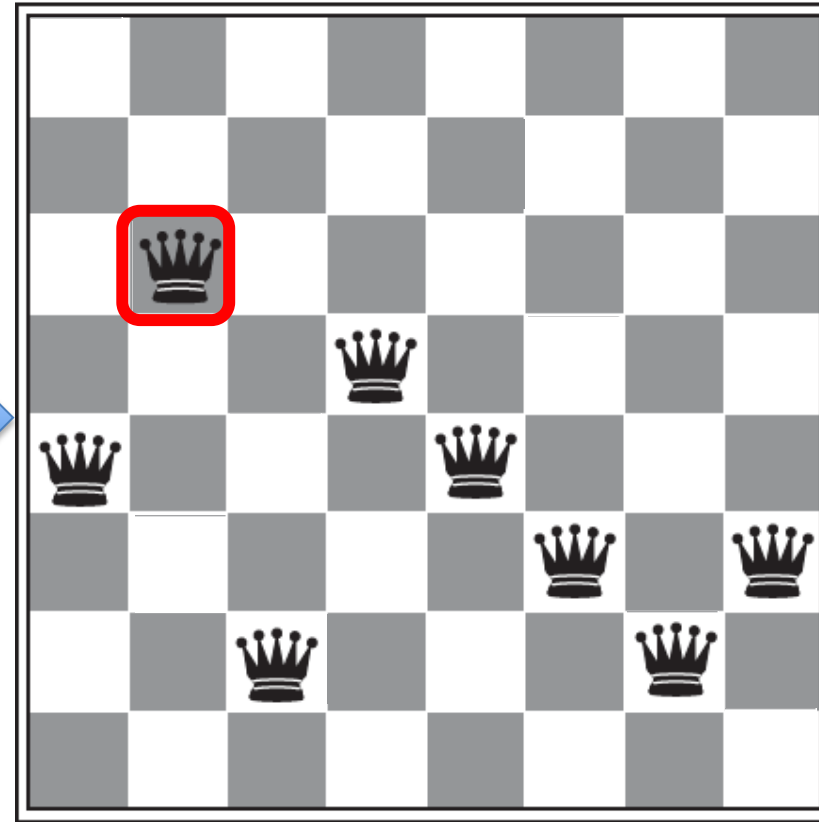


Hill climbing pentru problema celor 8 regine

Stare actuală



Posibil succesori



- $h = \text{\#perechi regine pe aceeași coloană} + \text{\#perechi regine pe aceeași linie} + \text{\#perechi regine pe aceeași diagonală} = 0 + 3 + 9 = 12$

Hill climbing pentru problema celor 8 regine

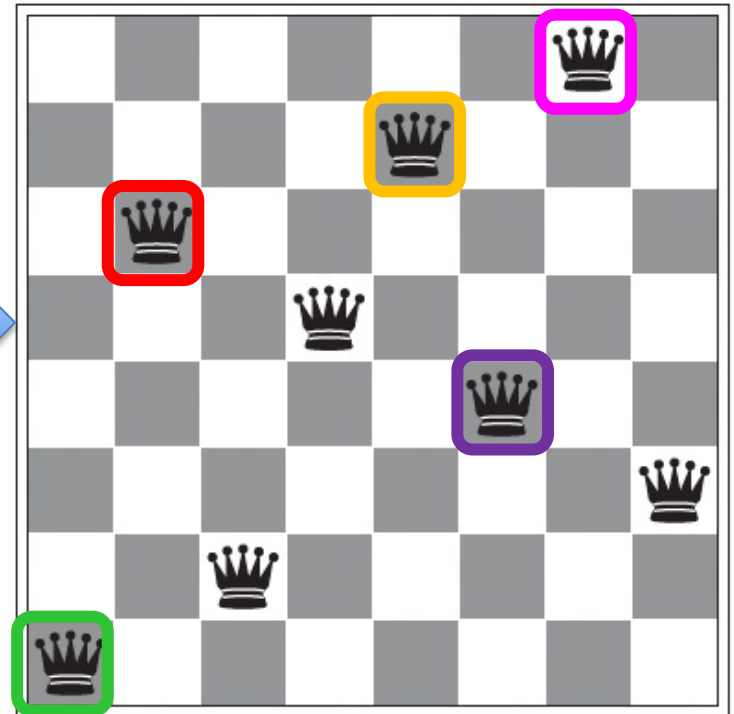
- sunt 56 de succesori pentru stare actuală
- calculăm funcția h pentru fiecare succesor
- există 8 succesori cu $h = 12$
- hill climbing alege aleator între cei 8 succesori
- hill climbing = căutare locală greedy = alege un succesor optim local fără să aibă o privire de ansamblu globală

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

Hill climbing pentru problema celor 8 regine

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

5 mutări

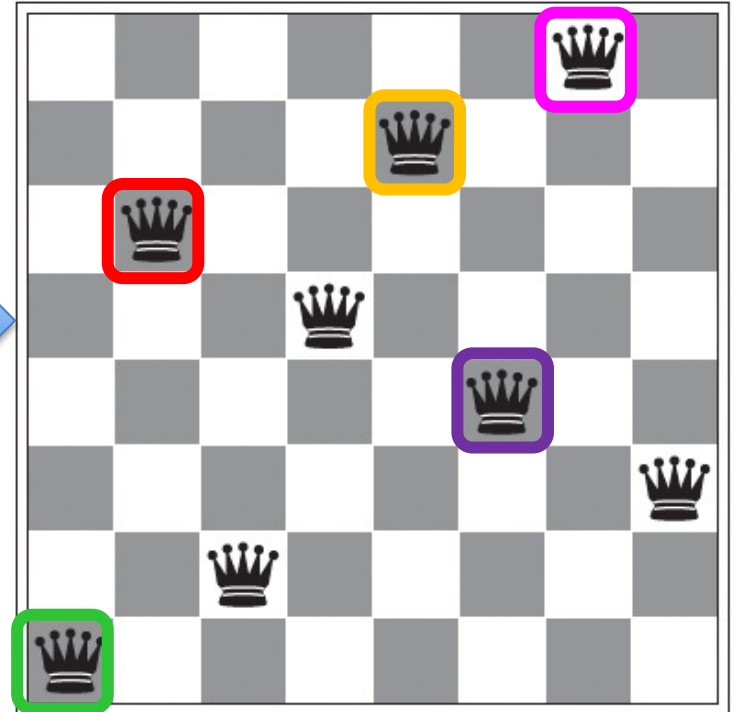


- hill climbing = căutare locală greedy = alege un succesori optim local fără să aibă o privire de ansamblu globală
- realizează un progres foarte rapid către o posibilă soluție ($h = 1$) întrucât e foarte ușor să îmbunătățim starea curentă (foarte proastă, $h = 17$)
- starea la care se ajunge ($h = 1$) este stare de minim local, nu este minim global. Nu mai poate fi îmbunătățită, dar nu am ajuns la o soluție ($h = 0$)

Hill climbing pentru problema celor 8 regine

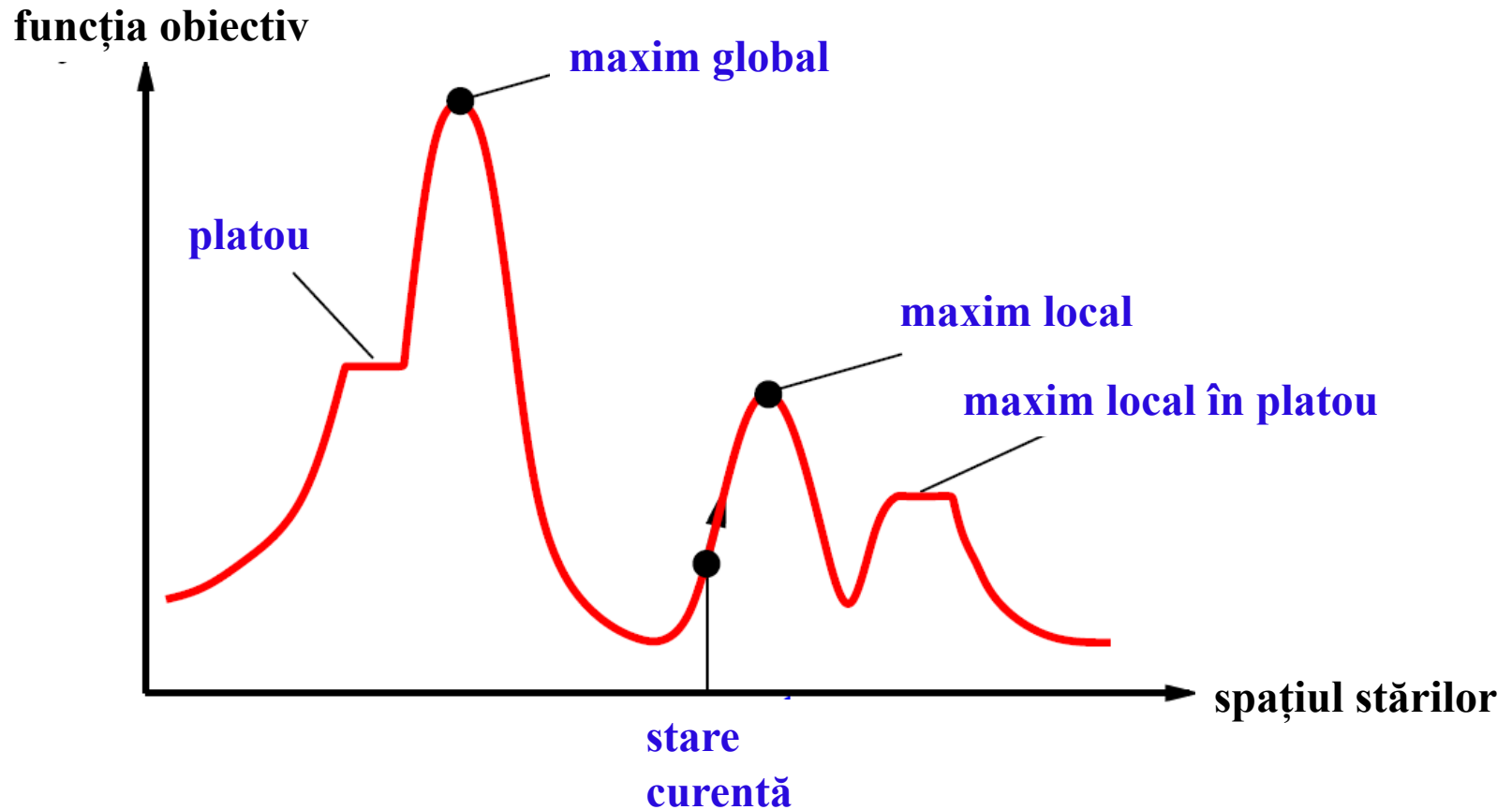
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

5 mutări



- pe problema celor 8 regine, pornind de la o stare aleatoare, hill climbing găsește o soluție globală ($h = 0$) în 14% din cazuri în medie în 4 mutări. Când se blochează într-un minim local o face în 86% din cazuri în medie în 3 mutări.
- spațiul stărilor are $8^8 \approx 17,000,000$ configurații posibile

Diagramă pentru Hill climbing

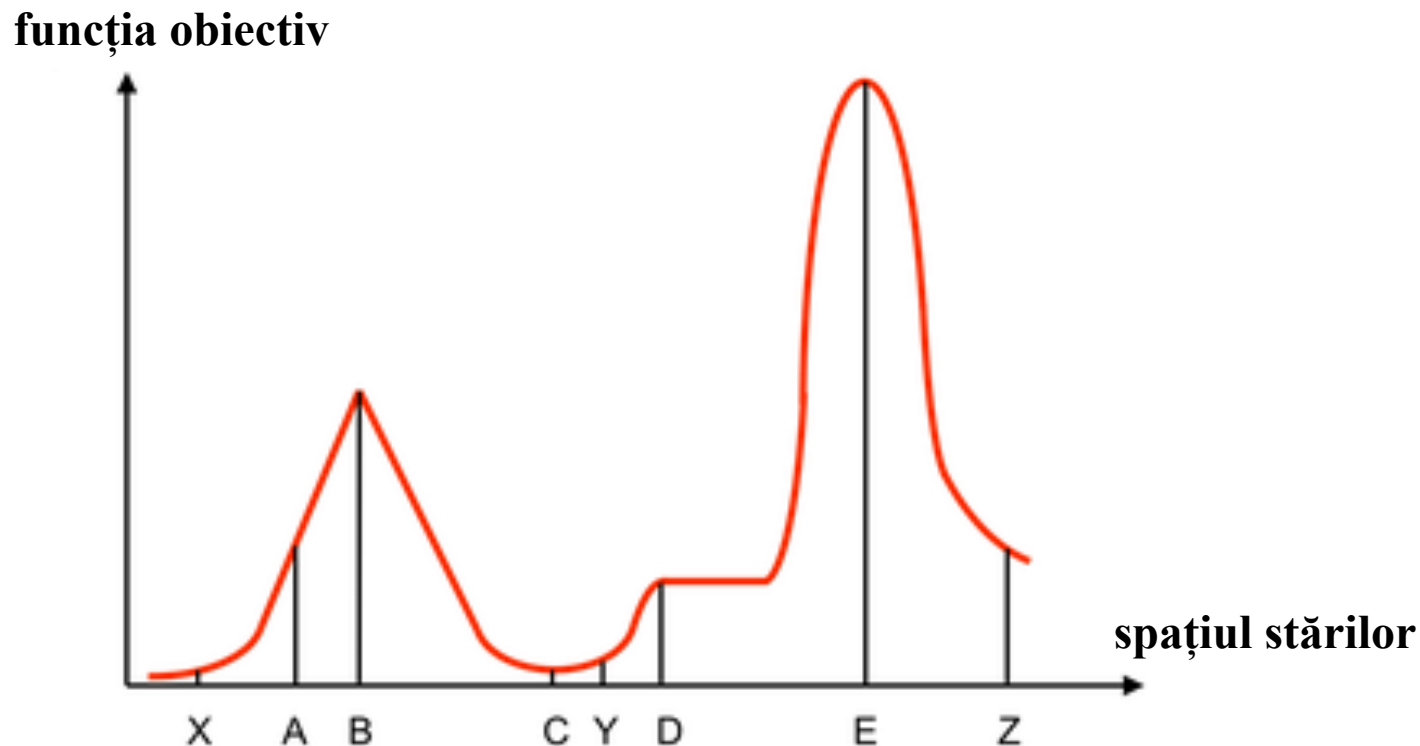


Problema de optim local: maxim global vs maxim local

- inițializări multiple în paralel + hill climbing + ia maximumul după toate rulările
- evadează din punctele de maxim local cu anumite metode de a explora spațiul –

algoritmi genetici, simulated annealing, etc.

Instanțieri pentru Hill climbing



Dacă starea curentă este X, unde ajungem ?

Dacă starea curentă este Y, unde ajungem ?

Dacă starea curentă este Z, unde ajungem ?

Calcul evolutiv – motivație și idee generală

În multe probleme de optimizare:

- spațiul soluțiilor este foarte mare;
- funcția cost/obiectiv prezintă multe minime/maxime locale;

Avem nevoie de o metode care să exploreze simultan mai multe regiuni ale spațiului soluțiilor.

Ideea centrală a **calculului evolutiv**

- menținem o **populație de soluții candidat**;
- evaluăm fiecare soluție printr-o **funcție de fitness**;
- selectăm preferențial soluțiile cele mai bune;
- generăm soluții noi prin variație (**încrucișare și mutație**);
- repetăm procesul până când este îndeplinită o condiție de oprire (calitatea soluțiilor devine satisfăcătoare, am atins un număr maxim de generații, etc)

Calculul evolutiv echilibrează:

- **explorarea** (cautarea de regiuni noi din spațiul soluțiilor);
- **exploatarea** (rafinarea soluțiile bune).

Acest „duet” permite rezolvarea unor probleme unde metodele deterministe sau greedy pot eșua.

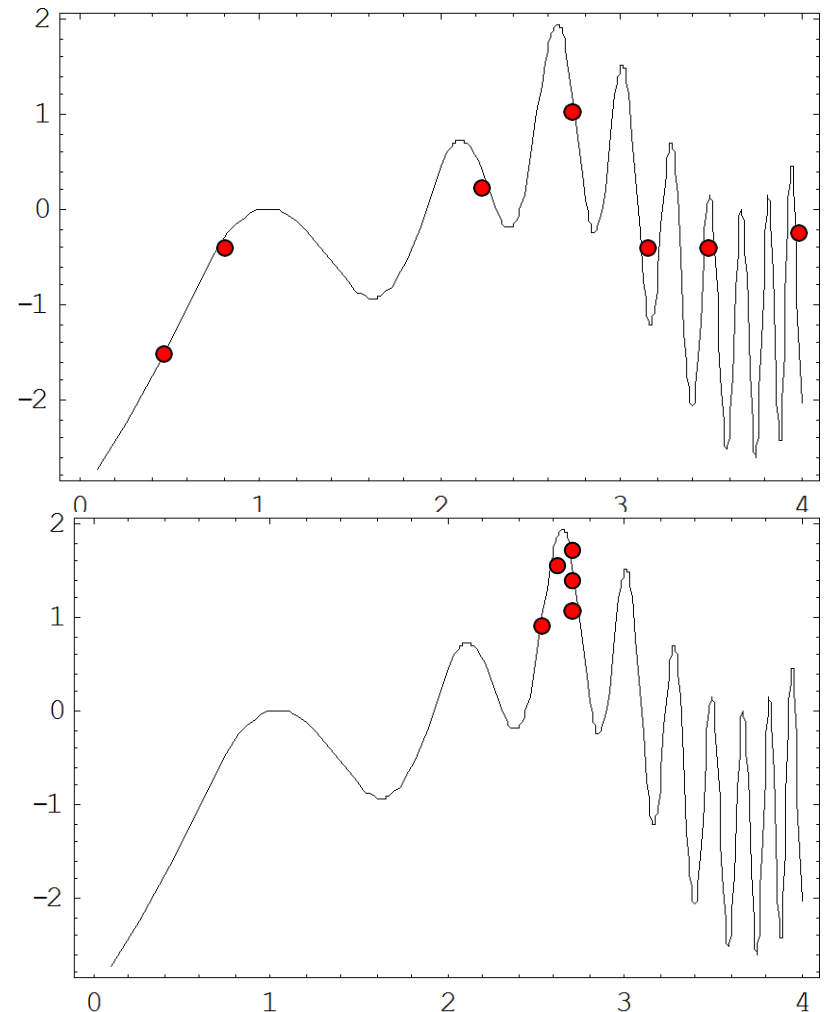
Specificul calculului evolutiv

Procesul de căutare se bazează pe două mecanisme principale:

explorare = investigarea diferitelor regiuni din spațiul soluțiilor pentru a descoperi soluții promițătoare

exploatare = rafinarea soluțiilor promițătoare (folosind informațiile acumulate în explorare)

Este necesar un echilibru între explorare și exploatare.

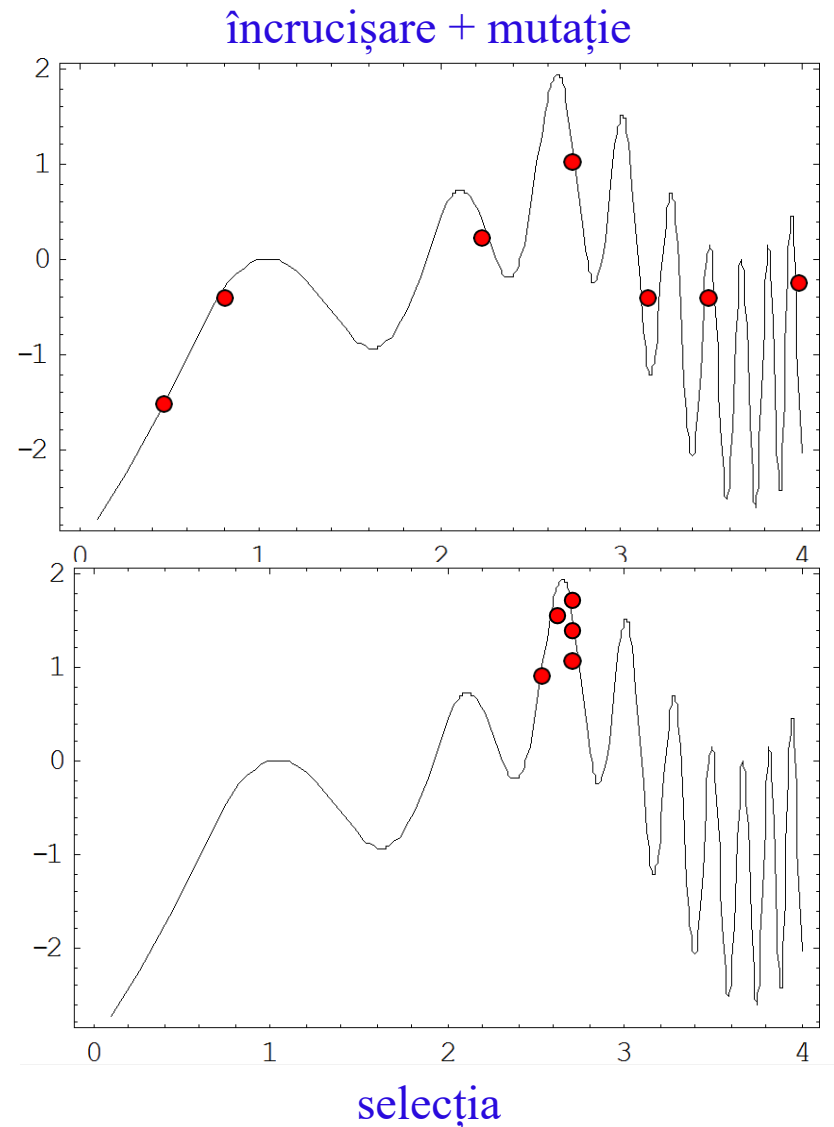


Calculul evolutiv → algoritmi evolutivi

Acest echilibru dintre explorare și exploatare este implementat în practică prin algoritmi evolutivi.

Algoritmul evolutiv este mecanismul concret prin care implementăm explorarea și exploatarea:

- variația (**încrucișare + mutație**) realizează explorarea
- **selecția** realizează exploatarea.



Structura unui algoritm evolutiv

Inițializare populație (aleatoare)

REPETĂ

Evaluarea populației (calcul fitness)

Selecția părinților

Generarea urmașilor prin încrucișare + mutație

Evaluarea urmașilor (calcul fitness)

Selecția supraviețuitorilor (din părinți și/sau urmași)

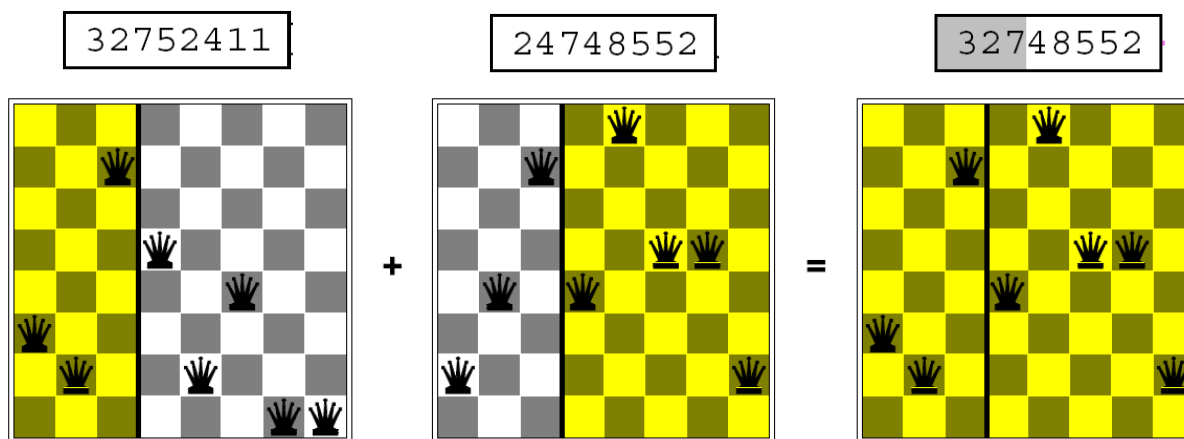
PÂNĂ CÂND <condiție de oprire>

Tipuri de algoritmi evolutivi

Algoritmii evolutivi diferă în principal prin modul de reprezentare a soluțiilor:

- Șiruri binare (codificare discretă)
→ Algoritmi genetici
- Vectori de valori reale
→ Strategii evolutive
- Structuri de tip mașină de stări / programe
→ Programare evolutivă
- Structuri de tip arbore (expresii, programe)
→ Programare genetică

Algoritmi genetici – exemplu



Populație = mulțime de indivizi (ipoteze, stări, asignări, configurări)

Funcție de evaluare (fitness) = evaluează calitatea unei ipoteze

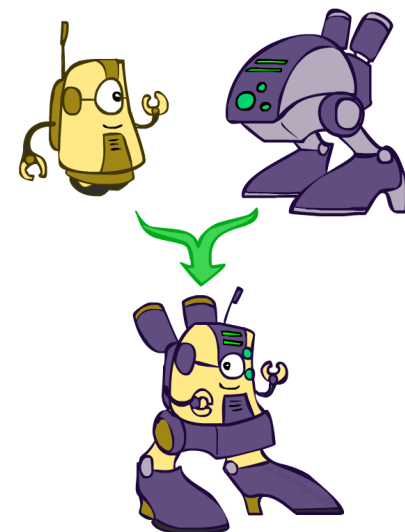
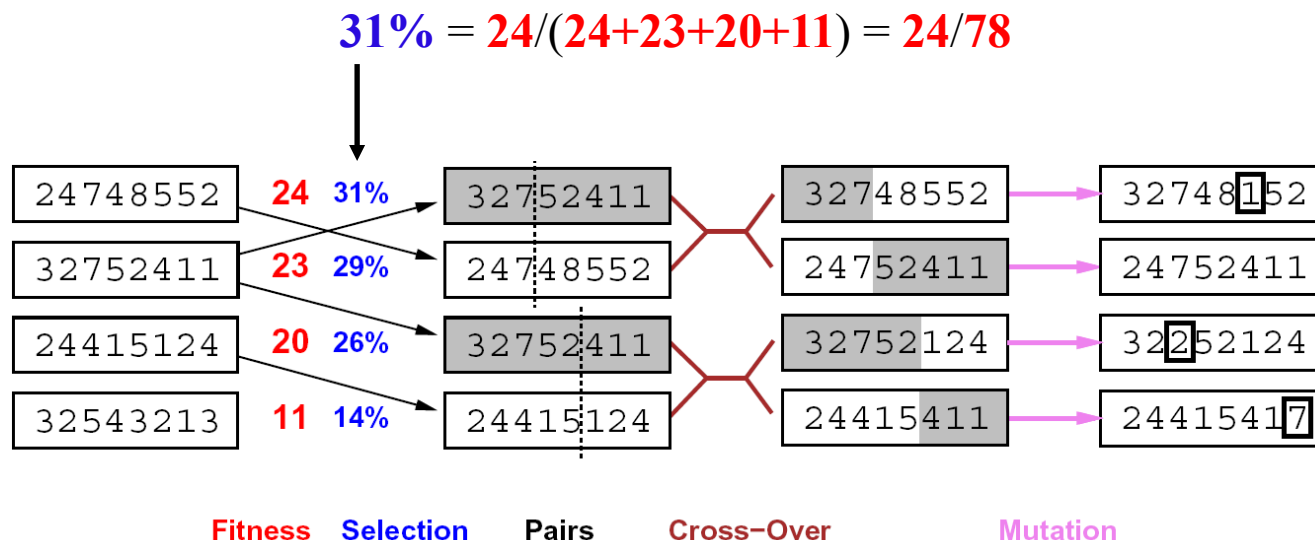
-exemplu în problema de N-regine: numărul de perechi de regine care nu se atacă

$$f(32752411) = 23 = 28 - 5$$

$$f(24748552) = 24 = 28 - 4$$

Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție după o anumită distribuție de probabilitate și împerechere bazată pe operațiile de încrucișare (cross-over) și mutație.

Algoritmi genetici – exemplu



- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (**selection**) după o anumită distribuție de probabilitate bazată pe funcția de evaluare (**fitness**) și împerechere (**pairs**) bazată pe operațiile de **încrucișare (cross-over)** și mutație (**mutation**)

Algoritmi genetici – exemplu

